

TEST 1

Pytanie 1: Które miary są używane do oceny wydajności obliczeń równoległych?

Poprawne odpowiedzi:

- Przyspieszenie
- Efektywność
- Narzut równoległego wykonania
- Skalowalność

Raczej nie jako podstawowe miary wydajności:

- **Ilość użytych semaforów w programie** — dotyczy synchronizacji, ale sama w sobie nie jest miarą wydajności.
- **Liczba wykonywanych wątków** — opisuje konfigurację programu, ale nie mówi bezpośrednio, jak dobrze program działa równolegle.

Pojęcia do notatki

Przyspieszenie

Przyspieszenie określa, ile razy szybciej program działa w wersji równoległej w porównaniu do wersji sekwencyjnej.

Najczęściej zapisuje się je wzorem:

$$S_p = \frac{T_1}{T_p}$$

gdzie:

- T_1 — czas wykonania programu na jednym procesorze lub w jednym wątku,
- T_p — czas wykonania programu na p procesorach lub wątkach,
- S_p — przyspieszenie dla p jednostek wykonawczych.

Przykład:

Jeśli program sekwencyjny działa 100 sekund, a równoległy na 4 wątkach działa 25 sekund, to:

$$S_4 = \frac{100}{25} = 4$$

czyli program przyspieszył 4 razy.

Efektywność

Efektywność mówi, jak dobrze wykorzystywane są procesory, rdzenie lub wątki.

Wzór:

$$E_p = \frac{S_p}{p}$$

gdzie:

- S_p — przyspieszenie,
- p — liczba procesorów, rdzeni lub wątków.

Efektywność często podaje się jako liczbę od 0 do 1 albo jako procent.

Przykład:

Jeśli dla 4 wątków przyspieszenie wynosi 2, to:

$$E_4 = \frac{2}{4} = 0,5 = 50\%$$

Oznacza to, że zasoby są wykorzystywane tylko w połowie.

Narzut równoległego wykonania

Narzut równoległego wykonania to dodatkowy koszt wynikający z użycia obliczeń równoległych.

Może obejmować między innymi:

- tworzenie i kończenie wątków,
- komunikację między procesami,
- synchronizację,
- oczekiwanie na blokady,
- nierównomierny podział pracy,
- konflikty dostępu do danych.

Program równoległy nie zawsze działa szybciej, ponieważ część czasu jest tracona właśnie na narzut.

Skalowalność

Skalowalność określa, czy program nadal dobrze przyspiesza po zwiększeniu liczby procesorów, rdzeni lub wątków.

Program jest dobrze skalowalny, jeśli po dodaniu większej liczby jednostek obliczeniowych jego czas wykonania nadal wyraźnie maleje.

Przykład:

Jeśli program na 2 wątkach działa prawie 2 razy szybciej, na 4 prawie 4 razy szybciej, a na 8 prawie 8 razy szybciej, to ma dobrą skalowalność.

Najważniejsza intuicja

Wydajność obliczeń równoległych oceniamy nie tylko po tym, czy program działa szybciej, ale też po tym, jak dobrze wykorzystuje dodatkowe zasoby i ile kosztuje synchronizacja oraz komunikacja.

Pytanie 2: Jakie są kluczowe zalety konteneryzacji w środowiskach HPC i chmurze?

Poprawne odpowiedzi:

- **Lekkość i szybkie uruchamianie**
- **Przenośność aplikacji niezależnie od infrastruktury**
- **Współdzielenie jądra systemu między kontenerami a systemem hosta**

Raczej niepoprawne:

- **Wymóg posiadania dedykowanego sprzętu** — kontenery właśnie zmniejszają zależność od konkretnego sprzętu i środowiska.
- **Możliwość współdzielenia kontenerów między użytkownikami w czasie rzeczywistym** — kontenery można udostępniać jako obrazy, ale współdzielenie jednego działającego kontenera w czasie rzeczywistym nie jest typową zaletą.
- **Każdy kontener wymaga osobnej licencji systemu operacyjnego** — kontener nie uruchamia pełnego osobnego systemu operacyjnego, więc nie wymaga osobnej licencji OS tak jak maszyna wirtualna.

Pojęcia do notatki

Konteneryzacja

Konteneryzacja to technika uruchamiania aplikacji wraz z jej zależnościami w odizolowanym środowisku zwanym **kontenerem**.

Kontener zawiera zwykle:

- aplikację,
- biblioteki,
- pliki konfiguracyjne,
- zależności wymagane do działania programu.

Nie zawiera jednak pełnego osobnego systemu operacyjnego, tak jak maszyna wirtualna.

Lekkość kontenerów

Kontenery są **lekkie**, ponieważ współdzielą jądro systemu operacyjnego hosta.

Oznacza to, że nie trzeba uruchamiać całego osobnego systemu operacyjnego dla każdej aplikacji.

Dzięki temu kontenery:

- zużywają mniej pamięci niż maszyny wirtualne,
 - uruchamiają się szybciej,
 - łatwiej je skalować,
 - są wygodne w środowiskach chmurowych i HPC.
-

Szybkie uruchamianie

Kontenery startują zwykle znacznie szybciej niż maszyny wirtualne, ponieważ nie muszą uruchamiać pełnego systemu operacyjnego.

W HPC i chmurze jest to ważne, bo można szybko uruchamiać wiele zadań obliczeniowych lub usług.

Przenośność aplikacji

Przenośność oznacza, że aplikację zamkniętą w kontenerze można uruchomić na różnych środowiskach bez dużych zmian.

Ten sam kontener może działać na przykład:

- na komputerze programisty,
- na klastrze HPC,
- w chmurze,
- na serwerze produkcyjnym.

Dzięki temu zmniejsza się problem:
„u mnie działa, a na serwerze nie”.

Współdzielenie jądra systemu

Kontenery korzystają ze wspólnego **jądra systemu operacyjnego** razem z systemem hosta.

To różni je od maszyn wirtualnych, które mają własny pełny system operacyjny.

Współdzielenie jądra daje:

- mniejszy narzut,
- szybsze działanie,
- mniejsze zużycie zasobów.

Jednocześnie wymaga mechanizmów izolacji, aby kontenery nie przeszkadzały sobie nawzajem.

HPC

HPC, czyli **High Performance Computing**, oznacza obliczenia wysokiej wydajności.

Dotyczy to środowisk, w których wykonuje się bardzo wymagające obliczenia, na przykład:

- symulacje naukowe,
- obliczenia numeryczne,
- analizę dużych zbiorów danych,
- modelowanie klimatu,
- obliczenia inżynierskie.

W HPC kontenery pomagają łatwiej przenosić aplikacje między klastrami i kontrolować wersje bibliotek.

Chmura obliczeniowa

Chmura obliczeniowa to środowisko, w którym zasoby takie jak procesory, pamięć, dysk i sieć są udostępniane przez dostawcę jako usługa.

Kontenery są w chmurze popularne, ponieważ można je łatwo:

- uruchamiać,
- kopiować,
- skalować,
- aktualizować,
- przenosić między środowiskami.

Najważniejsza intuicja

Kontenery są lekkie, szybkie i przenośne, bo izolują aplikację wraz z zależnościami, ale nie uruchamiają pełnego osobnego systemu operacyjnego

Pytanie 3: Jakie są główne kroki metodologii PCAM w programowaniu równoległym?

Poprawna odpowiedź:

- **Podział, Komunikacja, Agregacja, Odwzorowanie**

Pojęcia do notatki

Metodologia PCAM

PCAM to metodologia projektowania programów równoległych. Nazwa pochodzi od czterech etapów:

$$P \rightarrow C \rightarrow A \rightarrow M$$

czyli:

- **Partitioning** — podział,
- **Communication** — komunikacja,
- **Agglomeration** — agregacja,
- **Mapping** — odwzorowanie.

Służy do systematycznego zaprojektowania programu równoległego: od rozbicia problemu na części aż po przypisanie tych części do procesorów lub wątków.

P — Podział

Podział polega na rozbiciu problemu na mniejsze zadania, które mogą być wykonywane równolegle.

Można dzielić:

- **dane**, na przykład tablicę na fragmenty,
- **zadania**, na przykład różne etapy obliczeń,
- **obszary problemu**, na przykład siatkę obliczeniową w symulacji.

Celem jest znalezienie jak największej liczby niezależnych fragmentów pracy.

C — Komunikacja

Komunikacja określa, jakie dane muszą być wymieniane między zadaniami.

Po podziale problemu trzeba ustalić:

- które zadania są od siebie zależne,
- jakie dane muszą przekazywać,
- jak często muszą się komunikować,
- czy potrzebna jest synchronizacja.

Im mniej komunikacji, tym zwykle lepsza wydajność programu równoległego.

A — Agregacja

Agregacja polega na łączeniu mniejszych zadań w większe bloki pracy.

Robi się to po to, aby:

- zmniejszyć narzut zarządzania wieloma małymi zadaniami,
- ograniczyć liczbę komunikatów,
- poprawić lokalność danych,
- uzyskać lepszą wydajność.

Zbyt drobny podział może być nieoptyczny, bo program więcej czasu traci na komunikację i synchronizację niż na właściwe obliczenia.

M — Odwzorowanie

Odwzorowanie polega na przypisaniu zadań do konkretnych procesorów, rdzeni, procesów lub wątków.

Celem odwzorowania jest:

- równomierne obciążenie jednostek obliczeniowych,
- ograniczenie kosztów komunikacji,
- dobre wykorzystanie dostępnych zasobów.

Przykład:

Jeśli mamy 8 zadań i 4 rdzenie, można przypisać po 2 zadania do każdego rdzenia.

Najważniejsza intuicja

PCAM mówi, że najpierw dzielimy problem na części, potem określamy komunikację między nimi, następnie łączymy zbyt małe zadania, a na końcu przypisujemy je do procesorów lub wątków.

Pytanie 6: Wskaż poprawne stwierdzenia dotyczące punktów szeregowania zadań OpenMP.

Poprawne odpowiedzi:

- Punkt szeregowania występuje po / przy dyrektywie `#pragma omp taskwait`
- Punkt szeregowania może występować w barierze `#pragma omp barrier`
- Wątek może wznowić realizację zadania typu `untied` w dowolnym punkcie szeregowania

Niepoprawne odpowiedzi:

- Wszystkie zadania są wykonywane dokładnie w tej kolejności, w jakiej zostały utworzone
 - Zadania OpenMP zawsze wykonują się równolegle, niezależnie od wartości klauzuli `if`
 - Punkt szeregowania występuje przed wykonaniem sekcji krytycznej `#pragma omp critical`
-

Pojęcia do notatki

Punkt szeregowania zadania

Punkt szeregowania zadania, czyli **task scheduling point**, to miejsce, w którym wykonywane aktualnie zadanie może zostać zawieszone, a wątek może zacząć wykonywać inne zadanie albo wznowić inne zawieszone zadanie. Specyfikacja OpenMP opisuje taki punkt jako miejsce, w którym bieżący region zadania może zostać zawieszony i później wznowiony.

`#pragma omp taskwait`

Dyrektywa:

```
#pragma omp taskwait
```

powoduje oczekiwanie na zakończenie zadań potomnych utworzonych przez bieżące zadanie.

W kontekście szeregowania jest ważna, ponieważ w regionie taskwait występuje punkt szeregowania zadania. Specyfikacja OpenMP wymienia taskwait region jako jedno z miejsc, gdzie istnieją punkty szeregowania.

Przykład:

```
#pragma omp task
{
    oblicz_A();
}
```

```
#pragma omp task
{
    oblicz_B();
}
```

```
#pragma omp taskwait
// tutaj bieżące zadanie czeka na zakończenie wcześniejszych zadań potomnych
```

Bariera jako punkt szeregowania

Dyrektywa:

```
#pragma omp barrier
```

synchronizuje wątki, czyli każdy wątek musi poczekać, aż pozostałe wątki również dojdą do bariery.

W OpenMP punkt szeregowania może występować zarówno w **barierze jawnej**, czyli #pragma omp barrier, jak i w **barierze niejawnej**, na przykład na końcu niektórych regionów równoległych. Specyfikacja wymienia punkty szeregowania w regionie bariery jawnej i niejawnej.

Zadanie untied

Zadanie typu **untied** nie jest przypisane na stałe do jednego wątku.

Jeśli takie zadanie zostanie zawieszone w punkcie szeregowania, to później może zostać wznowione przez dowolny wątek z tego samego zespołu wątków. W OpenMP zadanie untied jest zdefiniowane jako takie, które po zawieszeniu może być wznowione przez dowolny wątek w zespole.

Przykład:

```
#pragma omp task untied
{
    wykonaj_prace();
}
```


To różni się od zadania **tied**, które po zawieszeniu musi zostać wznowione przez ten sam wątek.

Kolejność wykonywania zadań

Zadania OpenMP **nie muszą wykonywać się w kolejności, w jakiej zostały utworzone**.

Kolejność wykonania może zależeć od:

- dostępności wątków,
- zależności między zadaniami,
- punktów synchronizacji,
- decyzji środowiska wykonawczego OpenMP.

Specyfikacja mówi, że gdy kilka zadań jest dostępnych do wykonania, wybór wykonywanego zadania może być nieokreślony.

Klauzula if przy zadaniach

Zadania OpenMP nie zawsze wykonują się równolegle. Klauzula if może zdecydować, czy zadanie zostanie odłożone do późniejszego wykonania, czy wykonane od razu.

Przykład:

```
#pragma omp task if(n > 1000)
{
    oblicz();
}
```

Jeśli warunek if jest fałszywy, zadanie może zostać wykonane natychmiast przez bieżący wątek, czyli bez faktycznego równoległego odłożenia. Specyfikacja OpenMP podaje, że gdy if przy jawnym zadaniu ma wartość fałszywą, zadanie jest wykonywane natychmiast po utworzeniu.

Sekcja krytyczna critical

Dyrektywa:

```
#pragma omp critical
{
    // kod wykonywany tylko przez jeden wątek naraz
}
```

służy do ochrony fragmentu kodu przed jednoczesnym wykonaniem przez wiele wątków.

Sama dyrektywa critical nie jest jednak standardowo wymieniana jako miejsce występowania punktu szeregowania zadania. Punkty szeregowania są związane między innymi z task, taskwait, taskyield, taskgroup oraz barierami.

Najważniejsza intuicja

Punkt szeregowania to miejsce, w którym OpenMP może przełączyć wątek z jednego zadania na inne; typowe miejsca to taskwait, bariery i operacje związane z zadaniami, ale nie zwykle wejście do sekcji critical.

Pytanie 7: Które technologie są wykorzystywane w systemach kolejkowych do zarządzania obciążeniem?

Najbardziej poprawne odpowiedzi:

- **SLURM**
- **HTCondor**
- **PBS, czyli Portable Batch System**

W szerszym sensie może też pasować:

- **Kubernetes** — zarządza uruchamianiem zadań i kontenerów w klastrze, ale nie jest klasycznym systemem kolejkowym HPC.
- **Docker Swarm** — zarządza kontenerami w klastrze, ale również nie jest typowym systemem kolejkowym HPC.

Niepoprawna odpowiedź:

- **MPI, czyli Message Passing Interface** — to biblioteka/interfejs do komunikacji między procesami, a nie system kolejkowy.

Pojęcia do notatki

System kolejkowy

System kolejkowy to oprogramowanie, które zarządza uruchamianiem zadań obliczeniowych na klastrze.

Użytkownik nie uruchamia programu bezpośrednio na wybranym komputerze, tylko zgłasza zadanie do kolejki. System decyduje:

- kiedy zadanie zostanie uruchomione,
- na których węzłach klastra,
- ile procesorów, rdzeni, pamięci lub GPU dostanie,
- jak długo może działać,
- jakie zadania mają priorytet.

System kolejkowy pozwala wielu użytkownikom korzystać ze wspólnego klastra w uporządkowany sposób.

SLURM

SLURM to popularny system zarządzania zadaniami i zasobami w klastrach HPC.

Pozwala użytkownikom zgłaszać zadania do kolejki, na przykład za pomocą polecenia:

sbatch skrypt.sh

SLURM przydziela zasoby, uruchamia zadania i pilnuje ich wykonania.

Typowe funkcje SLURM:

- kolejkovanie zadań,
 - przydział węzłów i rdzeni,
 - obsługa zadań równoległych,
 - ustawianie limitów czasu,
 - monitorowanie stanu zadań.
-

HTCondor

HTCondor to system do zarządzania dużą liczbą zadań obliczeniowych, szczególnie w środowiskach rozproszonych.

Dobrze nadaje się do sytuacji, gdy mamy wiele niezależnych zadań, które można wykonywać na różnych maszynach.

Przykład zastosowania:

- analiza wielu plików danych,
 - uruchamianie wielu symulacji,
 - rozproszone przetwarzanie dużej liczby zadań.
-

PBS — Portable Batch System

PBS to klasyczny system kolejkowy używany w klastrach obliczeniowych.

Podobnie jak SLURM, pozwala zgłaszać zadania wsadowe, przydzielać zasoby i zarządzać kolejkami.

Przykładowe cechy PBS:

- obsługa kolejek zadań,
 - harmonogramowanie pracy,
 - przydział procesorów i pamięci,
 - kontrola czasu wykonania.
-

Kubernetes

Kubernetes to system orkiestracji kontenerów.

Nie jest typowym systemem kolejkowym HPC, ale również zarządza obciążeniem w klastrze. Decyduje, na których maszynach uruchomić kontenery i jak utrzymać ich działanie.

W kontekście chmury Kubernetes może pełnić podobną rolę zarządzania zasobami, ale bardziej dla aplikacji kontenerowych niż klasycznych zadań HPC.

Docker Swarm

Docker Swarm to narzędzie do zarządzania klastrem kontenerów Dockera.

Podobnie jak Kubernetes, zajmuje się uruchamianiem usług i rozdzielaniem ich między węzły klastra. Jest jednak mniej typowy dla klasycznych systemów kolejkowych HPC.

MPI

MPI, czyli **Message Passing Interface**, nie jest systemem kolejkowym.

MPI służy do komunikacji między procesami w programach równoległych. Na przykład procesy mogą przysyłać sobie wiadomości za pomocą funkcji takich jak:

`MPI_Send(...)`

`MPI_Recv(...)`

MPI może być używane razem z systemem kolejkowym, na przykład zadanie MPI może zostać uruchomione przez SLURM, ale samo MPI nie zarządza kolejką zadań.

Najważniejsza intuicja

SLURM, HTCondor i PBS to klasyczne systemy kolejkowe do zarządzania zadaniami w klastrach; Kubernetes i Docker Swarm zarządzają obciążeniem kontenerów, a MPI służy do komunikacji, nie do kolejkowania.

Pytanie 8: Jakie są kluczowe cechy obliczeń masowo równoległych?

Poprawne odpowiedzi:

- **Wykorzystują dużą liczbę wątków jednocześnie**
- **Najczęściej bazują na architekturze GPU**

Raczej niepoprawne:

- **Są stosowane głównie w systemach jednordzeniowych** — nie, obliczenia masowo równoległe wymagają wielu jednostek wykonawczych.
- **Wymagają globalnej pamięci współdzielonej** — nie zawsze; wiele systemów masowo równoległych korzysta z pamięci rozproszonej albo hierarchicznej.
- **Nie są stosowane w obliczeniach naukowych** — przeciwnie, są bardzo często stosowane w nauce.

- **Najczęściej bazują na architekturze CPU** — CPU może brać udział w takich obliczeniach, ale typowym przykładem masowej równoległości są GPU.
-

Pojęcia do notatki

Obliczenia masowo równoległe

Obliczenia masowo równoległe polegają na jednoczesnym wykonywaniu bardzo dużej liczby operacji przez wiele jednostek obliczeniowych.

Zamiast kilku wątków mamy często setki, tysiące albo nawet więcej lekkich wątków.

Najważniejsza idea: problem trzeba podzielić na bardzo wiele małych, podobnych zadań, które mogą być wykonywane jednocześnie.

Duża liczba wątków

W obliczeniach masowo równoległych wykorzystuje się ogromną liczbę wątków lub jednostek wykonawczych.

Przykład:

Zamiast liczyć elementy tablicy jeden po drugim, można przypisać osobny wątek do każdego elementu lub grupy elementów.

// idea: wiele wątków liczy różne elementy tablicy jednocześnie

$C[i] = A[i] + B[i];$

Taki model dobrze pasuje do zadań, gdzie wiele operacji jest podobnych i niezależnych.

GPU

GPU, czyli **Graphics Processing Unit**, to procesor graficzny.

GPU jest często używane w obliczeniach masowo równoległych, ponieważ zawiera bardzo dużo prostszych rdzeni/jednostek wykonawczych, które mogą wykonywać wiele podobnych operacji jednocześnie.

GPU dobrze sprawdza się w zadaniach takich jak:

- operacje na macierzach,
 - przetwarzanie obrazów,
 - symulacje fizyczne,
 - uczenie maszynowe,
 - obliczenia naukowe.
-

CPU a GPU

CPU jest bardziej uniwersalne i dobrze radzi sobie z różnorodnymi zadaniami, logiką programu i sterowaniem.

GPU jest mniej uniwersalne, ale bardzo wydajne, gdy trzeba wykonać tę samą lub podobną operację na wielu danych naraz.

Dlatego w obliczeniach masowo równoległych często CPU zarządza programem, a GPU wykonuje właściwe intensywne obliczenia.

Pamięć współdzielona i rozproszona

Obliczenia masowo równoległe nie muszą zawsze wymagać jednej globalnej pamięci współdzielonej.

W praktyce mogą występować różne modele pamięci:

- **pamięć lokalna** dla wątków,
- **pamięć współdzielona** w obrębie grupy wątków,
- **pamięć globalna** na GPU,
- **pamięć rozproszona** między węzłami klastra.

Dlatego stwierdzenie, że masowa równoległość wymaga globalnej pamięci współdzielonej, jest zbyt mocne.

Zastosowania naukowe

Obliczenia masowo równoległe są bardzo często stosowane w nauce i technice.

Przykłady:

- symulacje klimatu,
 - symulacje cząsteczek,
 - obliczenia numeryczne,
 - analiza dużych danych,
 - modelowanie zjawisk fizycznych,
 - sztuczna inteligencja.
-

Najważniejsza intuicja

Obliczenia masowo równoległe polegają na uruchomieniu bardzo wielu prostych operacji jednocześnie; najczęściej kojarzą się z GPU, bo GPU ma architekturę stworzoną do pracy na ogromnej liczbie wątków.

Pytanie 9: Które z poniższych cech dotyczą wątków jądra, czyli wątków systemowych?

Poprawne odpowiedzi:

- Są zarządzane bezpośrednio przez system operacyjny
- Mają bezpośredni dostęp do zasobów sprzętowych
- Są wolniejsze w kontekście przełączania kontekstu niż wątki użytkownika

Niepoprawne odpowiedzi:

- **Działają wyłącznie w środowisku JVM** — nie, wątki jądra są mechanizmem systemu operacyjnego, niezależnym od JVM.
- **Nie mogą wykonywać operacji systemowych** — przeciwnie, są obsługiwane przez system operacyjny i mogą wykonywać operacje systemowe.
- **Nie współdzielą zasobów z innymi wątkami** — wątki jednego procesu zwykle współdzielą przestrzeń adresową i zasoby procesu.

Pojęcia do notatki

Wątek jądra / wątek systemowy

Wątek jądra, inaczej **wątek systemowy**, to wątek zarządzany bezpośrednio przez **jądro systemu operacyjnego**.

System operacyjny decyduje między innymi:

- kiedy dany wątek ma być uruchomiony,
- na którym rdzeniu procesora będzie wykonywany,
- kiedy zostanie zatrzymany,
- kiedy nastąpi przełączenie na inny wątek.

Zarządzanie przez system operacyjny

Wątki jądra są widoczne dla systemu operacyjnego.

Oznacza to, że systemowy planista zadań może je przydzielać do różnych rdzeni procesora.

Dzięki temu wątki jądra mogą rzeczywiście wykonywać się równolegle na wielu rdzeniach.

Dostęp do zasobów sprzętowych

Wątki jądra działają pod kontrolą systemu operacyjnego i mogą korzystać z mechanizmów systemowych, takich jak:

- operacje wejścia-wyjścia,
- dostęp do plików,

- komunikacja sieciowa,
- synchronizacja,
- przydział czasu procesora,
- obsługa przerw i wywołań systemowych.

Nie oznacza to, że wątek samodzielnie omija system operacyjny i bez kontroli steruje sprzętem, ale ma dostęp do zasobów przez mechanizmy jądra.

Przełączanie kontekstu

Przełączanie kontekstu to operacja, w której procesor przestaje wykonywać jeden wątek i zaczyna wykonywać inny.

Przy przełączaniu trzeba zapisać stan aktualnego wątku i odtworzyć stan kolejnego.

W przypadku wątków jądra przełączanie kontekstu jest zwykle droższe niż przy wątkach użytkownika, ponieważ wymaga udziału systemu operacyjnego.

Wątki użytkownika

Wątki użytkownika są zarządzane przez bibliotekę lub środowisko uruchomieniowe w przestrzeni użytkownika, a nie bezpośrednio przez jądro systemu.

Ich przełączanie może być szybsze, bo nie zawsze wymaga interwencji systemu operacyjnego.

Minusem jest to, że jeśli system operacyjny widzi tylko jeden wątek jądra, to wiele wątków użytkownika może nie korzystać w pełni z wielu rdzeni.

Współdzielenie zasobów przez wątki

Wątki należące do tego samego procesu zwykle współdzielą:

- pamięć procesu,
- otwarte pliki,
- zmienne globalne,
- uchwyty do zasobów,
- przestrzeń adresową.

Każdy wątek ma natomiast własny stos i własny stan wykonania, na przykład licznik instrukcji i rejestry.

Najważniejsza intuicja

Wątki jądra są „prawdziwie” widoczne dla systemu operacyjnego, więc mogą być planowane na wielu rdzeniach, ale ich przełączanie jest droższe niż w przypadku lekkich wątków użytkownika.

Pytanie 10: Które z poniższych cech są charakterystyczne dla systemów Grid Computing?

Poprawne odpowiedzi:

- **Skalowalność i heterogeniczność**
- **Współdzielenie zasobów między użytkownikami**

Niepoprawne odpowiedzi:

- **Wymóg jednolitej architektury sprzętowej** — Grid Computing może łączyć różne typy komputerów, systemów i sieci.
 - **Brak potrzeby synchronizacji między węzłami** — synchronizacja i koordynacja często są potrzebne.
 - **Praca wyłącznie w chmurze obliczeniowej** — grid może działać poza chmurą, na przykład między instytucjami naukowymi.
 - **Ograniczenie do zastosowań korporacyjnych** — grid jest często używany także w nauce, badaniach i dużych projektach obliczeniowych.
-

Pojęcia do notatki

Grid Computing

Grid Computing to model obliczeń rozproszonych, w którym wiele niezależnych komputerów, klastrów lub ośrodków obliczeniowych współpracuje nad wspólnymi zadaniami.

Zasoby mogą należeć do różnych organizacji i znajdować się w różnych lokalizacjach geograficznych.

Przykład:

Kilka uczelni lub instytutów badawczych udostępnia część swoich zasobów obliczeniowych do wspólnego projektu naukowego.

Skalowalność

Skalowalność oznacza możliwość zwiększania mocy obliczeniowej systemu przez dodawanie kolejnych zasobów.

W Grid Computing można dołączać nowe:

- komputery,
- klastry,
- serwery,
- pamięci masowe,
- ośrodki obliczeniowe.

Dzięki temu system może obsługiwać coraz większe problemy obliczeniowe.

Heterogeniczność

Heterogeniczność oznacza różnorodność zasobów w systemie.

W gridzie mogą współpracować maszyny różniące się:

- architekturą procesora,
- systemem operacyjnym,
- ilością pamięci,
- mocą obliczeniową,
- szybkością sieci,
- lokalizacją,
- oprogramowaniem.

Dlatego Grid Computing nie wymaga jednolitej architektury sprzętowej.

Współdzielenie zasobów

W Grid Computing zasoby są współdzielone między wielu użytkowników, projekty lub instytucje.

Współdzielone mogą być:

- procesory,
- pamięć,
- przestrzeń dyskowa,
- dane,
- aplikacje,
- specjalistyczne urządzenia.

Celem jest lepsze wykorzystanie dostępnej infrastruktury.

Synchronizacja i koordynacja

W systemach gridowych często potrzebna jest koordynacja między zadaniami i węzłami.

Może obejmować:

- podział pracy,
- przesyłanie danych,
- zbieranie wyników,
- kontrolę zależności,
- uwierzytelnianie użytkowników,

- zarządzanie dostępem do zasobów.

Nie zawsze każde zadanie wymaga ścisłej synchronizacji, ale nie można powiedzieć, że synchronizacja nigdy nie jest potrzebna.

Grid Computing a chmura

Grid Computing i chmura obliczeniowa są podobne, bo oba modele dotyczą korzystania z rozproszonych zasobów.

Różnica intuicyjna:

- **Grid Computing** — często federacja zasobów należących do różnych organizacji.
- **Cloud Computing** — zasoby dostarczane jako usługa przez konkretnego dostawcę chmury.

Grid nie musi działać wyłącznie w chmurze.

Najważniejsza intuicja

Grid Computing łączy wiele różnych, rozproszonych zasobów i pozwala współdzielić je między użytkownikami lub instytucjami, dlatego jego kluczowe cechy to skalowalność, heterogeniczność i współdzielenie zasobów.

Pytanie 11: Jakie operacje można wykonać na plikach w MPI-IO?

Poprawne odpowiedzi:

- `MPI_File_open()`
- `MPI_File_read()`
- `MPI_File_write_at()`
- `MPI_File_close()`

Niepoprawna odpowiedź:

- `MPI_File_lock()` — taka standardowa funkcja nie należy do typowych operacji MPI-IO. W MPI istnieje na przykład `MPI_Win_lock()` dla jednostronnej komunikacji RMA, ale nie jest to operacja na plikach MPI-IO.
-

Pojęcia do notatki

MPI-IO

MPI-IO to część standardu MPI służąca do równoległych operacji wejścia-wyjścia na plikach.

Pozwala wielu procesom MPI czytać i zapisywać dane do tego samego pliku w sposób skoordynowany.

Jest używane szczególnie w programach HPC, gdzie wiele procesów generuje lub przetwarza duże ilości danych.

MPI_File_open()

Funkcja:

MPI_File_open(...)

służy do otwarcia pliku w MPI-IO.

Plik może być otwierany przez wiele procesów jednocześnie, na przykład w celu wspólnego odczytu lub zapisu.

Przykładowe tryby otwarcia:

MPI_MODE_RDONLY
MPI_MODE_WRONLY
MPI_MODE_RDWR
MPI_MODE_CREATE

MPI_File_read()

Funkcja:

MPI_File_read(...)

służy do odczytu danych z pliku.

Każdy proces może odczytywać dane z aktualnej pozycji w pliku, zgodnie z ustawionym widokiem pliku i wskaźnikiem pozycji.

MPI_File_write_at()

Funkcja:

MPI_File_write_at(...)

służy do zapisu danych w konkretnym miejscu pliku, podanym jako przesunięcie, czyli **offset**.

Przykład intuicyjny:

Proces 0 zapisuje dane od początku pliku, proces 1 od dalszej pozycji, proces 2 jeszcze dalej.

Dzięki temu wiele procesów może zapisywać różne fragmenty jednego pliku.

MPI_File_close()

Funkcja:

MPI_File_close(...)

zamyka plik otwarty wcześniej przez MPI-IO.

Po zakończeniu operacji na pliku należy go zamknąć, aby system mógł poprawnie zakończyć zapis, zwolnić zasoby i uporządkować dane.

MPI_File_lock()

MPI_File_lock() nie jest standardową funkcją MPI-IO.

W MPI można spotkać funkcje z nazwą lock, na przykład:

MPI_Win_lock(...)

ale dotyczą one mechanizmu **RMA**, czyli zdalnego dostępu do pamięci, a nie operacji plikowych.

Najważniejsza intuicja

MPI-IO pozwala procesom MPI wspólnie otwierać, czytać, zapisywać i zamykać pliki; operacje blokowania typu MPI_File_lock() nie należą do standardowego zestawu funkcji MPI-IO.

Pytanie 12: Które z poniższych technologii są uniwersalne i mogą działać na różnych platformach sprzętowych?

Poprawne odpowiedzi:

- **OpenCL**
- **Vulkan**
- **TensorFlow**

Niepoprawne / mniej uniwersalne:

- **CUDA** — głównie dla kart NVIDIA.
 - **DirectX Compute Shaders** — związane z ekosystemem DirectX, głównie Windows/Xbox.
 - **PyCUDA** — Pythonowy interfejs do CUDA, więc również zależny od NVIDIA.
-

Pojęcia do notatki

OpenCL

OpenCL, czyli **Open Computing Language**, to otwarty standard do programowania równoległego na różnych typach urządzeń.

Może działać na:

- CPU,
- GPU,
- FPGA,
- akceleratorach obliczeniowych różnych producentów.

OpenCL jest uznawany za technologię bardziej uniwersalną niż CUDA, ponieważ nie jest ograniczony tylko do jednego producenta sprzętu.

Vulkan

Vulkan to niskopoziomowe API graficzne i obliczeniowe.

Może być używane na różnych platformach i przez różne układy graficzne. Vulkan obsługuje między innymi **compute shaders**, czyli shadery obliczeniowe, które pozwalają używać GPU nie tylko do grafiki, ale też do obliczeń ogólnego przeznaczenia.

Vulkan jest bardziej przenośny niż DirectX, ponieważ nie jest ograniczony wyłącznie do platform Microsoftu.

TensorFlow

TensorFlow to framework do uczenia maszynowego i obliczeń tensorowych.

Jest dość uniwersalny, ponieważ może działać na różnych typach sprzętu, na przykład:

- CPU,
- GPU,
- TPU,
- różnych systemach operacyjnych.

Nie jest jednak tak niskopoziomą technologią jak OpenCL czy Vulkan. TensorFlow bardziej ukrywa szczegóły sprzętu przed programistą.

CUDA

CUDA to platforma programowania równoległego opracowana przez NVIDIA.

Pozwala pisać programy działające na GPU, ale głównie na kartach graficznych NVIDIA.

Dlatego CUDA jest bardzo wydajna i popularna w HPC oraz AI, ale nie jest uniwersalna sprzętowo.

PyCUDA

PyCUDA to biblioteka pozwalająca używać CUDA z poziomu języka Python.

Ponieważ opiera się na CUDA, również jest zależna od kart NVIDIA.

Czyli:

PyCUDA → CUDA → NVIDIA GPU

DirectX Compute Shaders

DirectX Compute Shaders to mechanizm wykonywania obliczeń na GPU w ramach DirectX.

Pozwala pisać programy obliczeniowe dla GPU, ale jest mocno związany z platformą Microsoftu, szczególnie Windows i Xbox.

Dlatego nie jest tak uniwersalny jak OpenCL czy Vulkan.

Najważniejsza intuicja

Technologie uniwersalne nie są silnie przywiązane do jednego producenta sprzętu lub jednej platformy; dlatego OpenCL, Vulkan i TensorFlow są bardziej przenośne niż CUDA, PyCUDA czy DirectX Compute Shaders.

Pytanie 13: Które strategie mogą być używane do równoważenia obciążenia w programach równoległych?

Poprawne odpowiedzi:

- **Podział iteracji pętli, czyli Loop Partitioning**
- **Dynamiczny przydział pracy, czyli Task Pooling**

Raczej niepoprawne jako samodzielne strategie równoważenia obciążenia:

- **Wprowadzenie dodatkowej komunikacji między procesami** — komunikacja może być potrzebna do równoważenia obciążenia, ale sama w sobie nie jest strategią; może też zwiększać narzut.
 - **Zmniejszenie liczby rdzeni procesora** — zmniejsza równoległość, nie równoważy obciążenia.
 - **Wykorzystanie pamięci rozproszonej do synchronizacji** — dotyczy modelu pamięci i synchronizacji, a nie bezpośrednio balansowania pracy.
 - **Serializacja kodu równoległego** — usuwa równoległość, więc nie jest strategią równoważenia obciążenia.
-

Pojęcia do notatki

Równoważenie obciążenia

Równoważenie obciążenia, czyli **load balancing**, polega na takim podziale pracy między wątki, procesy lub węzły, aby każdy z nich miał podobną ilość pracy.

Celem jest uniknięcie sytuacji, w której jeden wątek nadal intensywnie pracuje, a pozostałe już skończyły i czekają.

Przykład problemu:

Wątek 1: 10 sekund pracy

Wątek 2: 10 sekund pracy

Wątek 3: 10 sekund pracy

Wątek 4: 60 sekund pracy

Cały program kończy się dopiero po 60 sekundach, mimo że większość wątków skończyła wcześniej.

Loop Partitioning

Loop Partitioning, czyli **podział iteracji pętli**, polega na rozdzieleniu iteracji pętli między wiele wątków lub procesów.

Przykład:

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    oblicz(i);
}
```

Jeżeli każda iteracja ma podobny koszt, można zastosować prosty podział statyczny, na przykład każdy wątek dostaje podobną liczbę iteracji.

Podział statyczny

Podział statyczny oznacza, że praca jest dzielona przed rozpoczęciem obliczeń.

Przykład:

Dla 100 iteracji i 4 wątków każdy wątek dostaje po 25 iteracji.

Zaleta:

- mały narzut organizacyjny.

Wada:

- działa słabo, jeśli różne iteracje mają różny czas wykonania.
-

Dynamiczny przydział pracy

Dynamiczny przydział pracy oznacza, że zadania są przydzielane w trakcie działania programu.

Wątek, który skończył swoją część pracy, pobiera kolejne zadanie z puli.

To pomaga, gdy nie wiadomo z góry, które zadania będą trwały długo, a które krótko.

Task Pooling

Task Pooling polega na utworzeniu puli zadań, z której wątki dynamicznie pobierają pracę.

Schemat:

Pula zadań:

[Zadanie 1] [Zadanie 2] [Zadanie 3] [Zadanie 4] ...

Wątek, który skończy pracę, pobiera następne dostępne zadanie.

Dzięki temu szybciej kończące wątki nie stoją bezczynnie, tylko wykonują kolejne zadania.

Narzut komunikacji

Czasem równoważenie obciążenia wymaga komunikacji między procesami, na przykład aby przekazać zadania z przeciążonego procesu do mniej obciążonego.

Nie należy jednak traktować samego „dodania komunikacji” jako celu. Komunikacja jest kosztem, który może pomóc w balansowaniu, ale jeśli jest jej za dużo, może pogorszyć wydajność.

Serializacja kodu

Serializacja kodu równoległego oznacza sprowadzenie wykonania do trybu sekwencyjnego lub ograniczenie równoległości.

Nie jest to metoda równoważenia obciążenia, ponieważ zamiast lepiej rozdzielać pracę między wiele jednostek, usuwa korzyści z równoległego wykonania.

Najważniejsza intuicja

Równoważenie obciążenia polega na tym, żeby wszystkie wątki lub procesy miały podobną ilość pracy; typowe metody to podział iteracji pętli oraz dynamiczne pobieranie zadań z puli.

Pytanie 14: Które z poniższych stwierdzeń są prawdziwe dla pamięci wspólnej?

Poprawne odpowiedzi:

- **Wszystkie procesory/wątki mają jednolitą przestrzeń adresową**
- **Umożliwia szybkie dzielenie danych między procesami**
- **Procesory muszą stosować synchronizację, aby uniknąć problemów współbieżności**

Niepoprawne odpowiedzi:

- **Wymaga komunikacji poprzez przesyłanie wiadomości** — to cecha pamięci rozproszonej, np. MPI.
 - **Każdy procesor/wątek ma swoją własną, niezależną pamięć** — to również bardziej pasuje do modelu pamięci rozproszonej.
 - **Jest wykorzystywana wyłącznie w systemach operacyjnych czasu rzeczywistego** — pamięć wspólna występuje powszechnie, nie tylko w systemach RTOS.
-

Pojęcia do notatki

Pamięć wspólna

Pamięć wspólna to model, w którym wiele procesorów, rdzeni lub wątków ma dostęp do tej samej przestrzeni adresowej.

Oznacza to, że różne wątki mogą odwoływać się do tych samych zmiennych i struktur danych.

Przykład:

```
int suma = 0;
```

```
#pragma omp parallel
{
    suma += 1;
}
```

Wszystkie wątki widzą zmienną suma, więc jest ona współdzielona.

Jednolita przestrzeń adresowa

Jednolita przestrzeń adresowa oznacza, że wątki korzystają ze wspólnego obszaru pamięci.

Jeśli jeden wątek zmieni wartość zmiennej, inne wątki mogą zobaczyć tę zmianę.

To upraszcza programowanie w porównaniu z przesyłaniem wiadomości, bo nie trzeba ręcznie wysyłać danych między procesami.

Dzielenie danych

Pamięć wspólna umożliwia szybkie dzielenie danych, ponieważ wiele wątków może korzystać z tych samych struktur danych bez jawnego przesyłania komunikatów.

Przykłady współdzielonych danych:

- tablice,
- zmienne globalne,
- kolejki zadań,
- bufora,
- struktury wyników.

To jest wygodne, ale wymaga ostrożności, bo wiele wątków może próbować modyfikować te same dane jednocześnie.

Synchronizacja

Synchronizacja jest potrzebna, aby uniknąć błędów wynikających z jednoczesnego dostępu do tych samych danych.

Typowe mechanizmy synchronizacji:

- muteksy,
- semafora,
- sekcje krytyczne,
- bariery,
- operacje atomowe.

Przykład problemu:

suma = suma + 1;

Ta operacja wygląda prosto, ale składa się z kilku kroków:

odczytaj suma

dodaj 1

zapisz suma

Jeśli dwa wątki wykonają te kroki jednocześnie, wynik może być błędny.

Race condition

Race condition, czyli **wyścig**, występuje wtedy, gdy wynik programu zależy od przypadkowej kolejności wykonania operacji przez wątki.

Przykład:

Dwa wątki jednocześnie zwiększają tę samą zmienną. Oba mogą odczytać starą wartość, a potem oba zapisać ten sam wynik, przez co jedno zwiększenie „ginie”.

Pamięć wspólna a przesyłanie wiadomości

W modelu pamięci wspólnej wątki komunikują się przez wspólne zmienne.

W modelu przesyłania wiadomości, na przykład w MPI, procesy mają osobne pamięci i muszą jawnie przysyłać dane:

```
MPI_Send(...);
```

```
MPI_Recv(...);
```

Dlatego zdanie, że pamięć wspólna wymaga przesyłania wiadomości, jest niepoprawne.

Najważniejsza intuicja

Pamięć wspólna ułatwia dzielenie danych, bo wiele wątków widzi te same zmienne, ale przez to trzeba stosować synchronizację, żeby uniknąć wyścigów i błędów współbieżności.

Pytanie 15: Jakie są główne wnioski wynikające z prawa Amdahla?

Poprawne odpowiedzi:

- Przy nieskończonej liczbie procesorów czas wykonania jest ograniczony przez część sekwencyjną programu
- Przyspieszenie programu nie może przekroczyć odwrotności udziału części sekwencyjnej
- Efektywność zrównoleglenia spada, gdy liczba procesorów rośnie

Niepoprawne odpowiedzi:

- Wydajność równoległa może być nieskończenie zwiększana niezależnie od kodu — nie, ograniczeniem jest część sekwencyjna.

- **Prawo Amdahla uwzględnia czas komunikacji między procesorami** — klasyczna postać prawa Amdahla nie uwzględnia jawnie narzutu komunikacji.
 - **Skalowalność systemu nie zależy od udziału części sekwencyjnej** — zależy bardzo mocno.
-

Pojęcia do notatki

Prawo Amdahla

Prawo Amdahla opisuje maksymalne możliwe przyspieszenie programu po jego zrównolegleniu.

Pokazuje, że nawet jeśli część programu wykonuje się równoległe bardzo szybko, to część sekwencyjna nadal ogranicza całkowity czas wykonania.

Wzór:

$$S_p = \frac{1}{s + \frac{1-s}{p}}$$

gdzie:

- S_p — przyspieszenie dla p procesorów,
 - s — część sekwencyjna programu,
 - $1 - s$ — część możliwa do zrównoleglenia,
 - p — liczba procesorów, rdzeni lub wątków.
-

Część sekwencyjna programu

Część sekwencyjna to fragment programu, którego nie da się wykonać równoległe.

Może to być na przykład:

- inicjalizacja danych,
- wczytywanie konfiguracji,
- fragment zależny od poprzednich wyników,
- operacje wymagające wykonania krok po kroku.

To właśnie ta część ogranicza maksymalne przyspieszenie programu.

Maksymalne przyspieszenie

Jeżeli liczba procesorów rośnie do nieskończoności, część równoległa może teoretycznie wykonać się prawie natychmiast.

Ale część sekwencyjna nadal pozostaje.

Dlatego maksymalne przyspieszenie wynosi:

$$S_{max} = \frac{1}{s}$$

Przykład:

Jeśli 10% programu jest sekwencyjne, czyli:

$$s = 0,1$$

to maksymalne przyspieszenie wynosi:

$$S_{max} = \frac{1}{0,1} = 10$$

Nawet przy ogromnej liczbie procesorów program nie będzie szybszy niż 10 razy.

Spadek efektywności

Efektywność obliczeń równoległych to:

$$E_p = \frac{S_p}{p}$$

Prawo Amdahla pokazuje, że po dodaniu kolejnych procesorów zysk jest coraz mniejszy.

Na początku dodatkowe procesory mogą mocno przyspieszać program, ale później większość czasu i tak ogranicza część sekwencyjna.

Dlatego efektywność zwykle spada, gdy liczba procesorów rośnie.

Narzut komunikacji

Klasyczne prawo Amdahla zakłada uproszczony model i nie opisuje dokładnie kosztów takich jak:

- komunikacja między procesorami,
- synchronizacja,
- oczekiwanie na dane,
- nierówny podział pracy.

W praktyce te narzuty mogą jeszcze bardziej ograniczać przyspieszenie.

Najważniejsza intuicja

Prawo Amdahla mówi, że programu nie da się przyspieszać w nieskończoność, bo zawsze ogranicza nas fragment, którego nie da się zrównoleglić.

16. Dla poniższego kodu: `#include <stdio.h> #include <omp.h> int main() { int id; // #pragma omp parallel private(id) #pragma omp parallel private(id) { id = omp_get_thread_num(); printf("Wątek %d przetwarza dane...\n", id); } return 0; }` Co można powiedzieć o sposobie przetwarzania i dostępu do zmiennej `id`?

Poprawne odpowiedzi:

- `id` jest prywatne dla każdego wątku, co oznacza, że każdy wątek ma swoją kopię.
- Jeśli nie użyto by klauzuli `private(id)`, to `id` byłoby współdzielone i powodowało błędy.
- Program może wykonać się poprawnie nawet bez `private(id)`, ale wynik mógłby być nieprzewidywalny.
- Wartość początkowa `id` nie jest zdefiniowana.

Niepoprawne odpowiedzi:

- `id` jest wspólne, więc każdy wątek nadpisuje jego wartość — nie w tym kodzie, bo użyto `private(id)`.
- `private(id)` oznacza, że `id` jest inicjalizowane domyślnie wartością 0 we wszystkich wątkach — nie, zmienna prywatna nie jest automatycznie zerowana.

Pojęcia do notatki

Klauzula `private` w OpenMP

Klauzula:

```
#pragma omp parallel private(id)
```

oznacza, że każdy wątek dostaje **własną kopię** zmiennej `id`.

Czyli zamiast jednej wspólnej zmiennej:

`id`

mamy osobne kopie:

`id` dla wątku 0

`id` dla wątku 1

`id` dla wątku 2

...

Dzięki temu wątki nie nadpisują sobie wzajemnie wartości.

Zmienna prywatna

Zmienna **prywatna** jest widoczna osobno dla każdego wątku.

W kodzie:

```
id = omp_get_thread_num();
```

każdy wątek zapisuje swój numer do swojej własnej kopii id.

Dlatego wypisanie:

```
printf("Wątek %d przetwarza dane...\n", id);
```

korzysta z lokalnej wartości danego wątku.

Zmienna współdzielona

Gdyby nie było:

```
private(id)
```

to zmienna id, zadeklarowana przed regionem równoległym, byłaby domyślnie **współdzielona**.

Wtedy wszystkie wątki zapisywałyby wartość do tej samej zmiennej:

wątek 0 zapisuje id = 0

wątek 1 zapisuje id = 1

wątek 2 zapisuje id = 2

...

Mogłoby dojść do sytuacji, w której jeden wątek zapisze swój numer, ale zanim go wypisze, inny wątek nadpisze zmienną.

To prowadzi do błędów współbieżności.

Race condition

Race condition, czyli **wyścig**, występuje wtedy, gdy wiele wątków jednocześnie odczytuje i zapisuje tę samą zmienną, a wynik zależy od przypadkowej kolejności wykonania.

Bez `private(id)` kod mógłby czasem wyglądać poprawnie, ale nie byłoby gwarancji poprawności.

Niezdefiniowana wartość początkowa

Zmienna oznaczona jako `private` nie przejmuje automatycznie wartości zmiennej oryginalnej.

Jej wartość początkowa jest **niezdefiniowana**, jeśli nie zostanie jawnie ustawiona.

W tym kodzie nie jest to problem, bo każdy wątek od razu przypisuje wartość:

```
id = omp_get_thread_num();
```

Gdybyśmy chcieli, aby zmienna prywatna dostała początkową wartość z oryginału, należałoby użyć `firstprivate`.

Najważniejsza intuicja

`private(id)` sprawia, że każdy wątek ma własne id, więc nie ma wyścigu przy zapisie numeru wątku; bez `private` zmienna byłaby wspólna i wynik mógłby być nieprzewidywalny.

Pytanie 17: Co przedstawia wykres roofline?

Poprawna odpowiedź:

- **Ograniczenie wydajnościowe wynikające z mocy obliczeniowej oraz przepustowości pamięci**

Niepoprawne odpowiedzi:

- **Zależności pomiędzy liczbą operacji I/O a czasem wykonania programu** — roofline dotyczy głównie relacji między obliczeniami a dostępem do pamięci.
- **Wydajność obliczeniową procesora w stosunku do jego zużycia energii** — to byłby raczej model energetyczny.
- **Porównanie czasu wykonania programu dla różnych poziomów optymalizacji kompilatora** — to zwykły wykres benchmarkowy, nie roofline.
- **Liczbę operacji zmiennoprzecinkowych wykonywanych na różnych typach procesorów** — roofline używa FLOPS jako miary wydajności, ale nie służy tylko do liczenia operacji.
- **Skalowanie programu równoległego w zależności od liczby procesorów** — to dotyczy np. wykresów skalowalności.

Pojęcia do notatki

Wykres roofline

Wykres roofline to model wydajności pokazujący, co ogranicza szybkość programu:

- **moc obliczeniowa procesora/GPU,**
- **przepustowość pamięci.**

Pomaga odpowiedzieć na pytanie:

czy program jest ograniczony przez obliczenia, czy przez dostęp do pamięci?

Oś X — intensywność arytmetyczna

Na osi poziomej znajduje się zwykle **intensywność arytmetyczna**, czyli:

$$\text{intensywność arytmetyczna} = \frac{\text{liczba operacji}}{\text{ilość przesłanych danych}}$$

Najczęściej wyrażana jako:

FLOP / bajt

Czyli: ile operacji zmiennoprzecinkowych program wykonuje na każdy bajt pobrany z pamięci.

Oś Y — wydajność

Na osi pionowej znajduje się **wydajność obliczeniowa**, najczęściej mierzona w:

FLOP/s

czyli liczbie operacji zmiennoprzecinkowych wykonywanych na sekundę.

Ograniczenie przez pamięć — memory-bound

Program jest **memory-bound**, gdy jego wydajność ogranicza szybkość pobierania danych z pamięci.

Dzieje się tak, gdy program wykonuje mało obliczeń na dużej ilości danych.

Przykład intuicyjny:

```
for (int i = 0; i < n; i++) {  
    c[i] = a[i] + b[i];  
}
```

Na każdy element wykonujemy mało obliczeń, ale trzeba dużo danych wczytać i zapisać.

Ograniczenie przez obliczenia — compute-bound

Program jest **compute-bound**, gdy jego wydajność ogranicza maksymalna moc obliczeniowa procesora lub GPU.

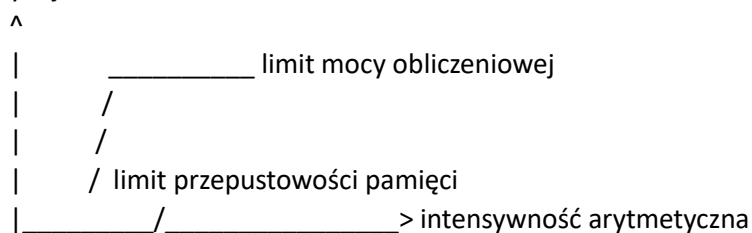
Dzieje się tak, gdy program wykonuje dużo operacji na stosunkowo małej ilości danych.

Przykładem mogą być intensywne obliczenia macierzowe, gdzie dane są wielokrotnie wykorzystywane.

Dach, czyli roof

Nazwa **roofline** pochodzi od kształtu wykresu przypominającego dach:

wydajność



Część ukośna oznacza ograniczenie przez pamięć, a część pozioma oznacza ograniczenie przez maksymalną moc obliczeniową.

Najważniejsza intuicja

Wykres roofline pokazuje, czy program działa wolno dlatego, że procesor/GPU ma za małą moc obliczeniową, czy dlatego, że dane są zbyt wolno dostarczane z pamięci.

Pytanie 18: Które z poniższych stwierdzeń są prawdziwe dla systemów MNSP, czyli **Multiple Nodes, Single Processor**?

Poprawne odpowiedzi:

- Każdy węzeł posiada jeden procesor
- Węzły komunikują się poprzez przesyłanie komunikatów
- Typowym środowiskiem programowania są PVM i MPI

Niepoprawne odpowiedzi:

- Procesory posiadają jednolitą przestrzeń adresową — to cecha pamięci współdzielonej, a nie systemu z wieloma oddzielnymi węzłami.
- Programowanie bazuje głównie na modelu pamięci współdzielonej — w MNSP dominuje model pamięci rozproszonej.
- Jest stosowany głównie w pojedynczych maszynach wielordzeniowych — MNSP dotyczy wielu węzłów, a nie jednej maszyny wielordzeniowej.

Pojęcia do notatki

MNSP — Multiple Nodes, Single Processor

MNSP oznacza architekturę, w której system składa się z wielu węzłów, a każdy węzeł ma jeden procesor.

Można to intuicyjnie przedstawić tak:

Węzeł 1: procesor + pamięć lokalna

Węzeł 2: procesor + pamięć lokalna

Węzeł 3: procesor + pamięć lokalna

...

Każdy węzeł działa jako osobna jednostka obliczeniowa.

Węzeł

Węzeł to pojedynczy komputer lub jednostka obliczeniowa w systemie rozproszonym.

W MNSP każdy węzeł ma:

- jeden procesor,
- własną pamięć lokalną,
- połączenie sieciowe z innymi węzłami.

Pamięć rozproszona

W systemach MNSP każdy węzeł posiada własną pamięć.

Oznacza to, że procesor z jednego węzła nie może bezpośrednio odwołać się do pamięci innego węzła tak jak do swojej lokalnej pamięci.

Dlatego nie ma jednej wspólnej przestrzeni adresowej dla wszystkich procesorów.

Przesyłanie komunikatów

Ponieważ węzły mają osobne pamięci, muszą wymieniać dane przez **komunikaty**.

Przykład:

Węzeł 1 wysyła dane do Węzła 2

Węzeł 2 odbiera dane i wykonuje dalsze obliczenia

W praktyce robi się to za pomocą bibliotek takich jak MPI lub PVM.

MPI

MPI, czyli **Message Passing Interface**, to standard programowania równoległego w systemach z pamięcią rozproszoną.

Typowe operacje MPI:

`MPI_Send(...);`

`MPI_Recv(...);`

Służą one do wysyłania i odbierania danych między procesami działającymi na różnych węzłach.

PVM

PVM, czyli **Parallel Virtual Machine**, to starsze środowisko do programowania obliczeń równoległych w systemach rozproszonych.

Pozwalało traktować wiele połączonych komputerów jak jedną „wirtualną maszynę równoległą”.

MNSP a maszyna wielordzeniowa

MNSP nie oznacza pojedynczego komputera z wieloma rdzeniami.

Pojedyncza maszyna wielordzeniowa ma zwykle wspólną pamięć i wiele rdzeni w jednym systemie.

MNSP oznacza wiele oddzielnych węzłów, które komunikują się przez sieć.

Najważniejsza intuicja

MNSP to system z wieloma oddzielnymi węzłami, gdzie każdy ma jeden procesor i własną pamięć, więc komunikacja odbywa się przez przesyłanie wiadomości, najczęściej z użyciem MPI lub PVM.

Pytanie 19: Jakie zalety posiada programowanie równoległe w modelu pamięci rozproszonej?

Poprawne odpowiedzi:

- **Umożliwia skalowalność systemu poprzez dodawanie nowych węzłów**
- **Pozwala na przetwarzanie danych na dużą skalę**
- **Umożliwia efektywne zarządzanie zasobami przez rozdzielanie obliczeń**
- **Pozwala na rozproszenie danych między różnymi komputerami**

Niepoprawne odpowiedzi:

- **Nie wymaga synchronizacji między węzłami** — synchronizacja często jest potrzebna, np. przy zbieraniu wyników albo wymianie danych.
- **Eliminuje konieczność stosowania przesyłania komunikatów** — przeciwnie, w pamięci rozproszonej komunikacja zwykle odbywa się przez przesyłanie komunikatów, np. MPI.

Pojęcia do notatki

Model pamięci rozproszonej

Pamięć rozproszona to model, w którym każdy procesor, proces lub węzeł posiada własną, lokalną pamięć.

Nie ma jednej wspólnej przestrzeni adresowej dla całego systemu.

Schematycznie:

Węzeł 1: CPU + pamięć lokalna

Węzeł 2: CPU + pamięć lokalna

Węzeł 3: CPU + pamięć lokalna

Jeśli jeden węzeł potrzebuje danych z drugiego, musi je otrzymać przez komunikację.

Skalowalność przez dodawanie węzłów

Jedną z największych zalet pamięci rozproszonej jest **skalowalność**.

Można zwiększać moc systemu przez dodawanie kolejnych komputerów lub węzłów do klastra.

Przykład:

4 węzły → mniejsza moc obliczeniowa

16 węzłów → większa moc obliczeniowa

64 węzły → jeszcze większa moc obliczeniowa

Dzięki temu model pamięci rozproszonej dobrze nadaje się do dużych systemów HPC.

Przetwarzanie danych na dużą skalę

Model pamięci rozproszonej pozwala przetwarzać dane, które są zbyt duże dla jednej maszyny.

Dane można podzielić między wiele komputerów.

Przykład:

Węzeł 1 przetwarza część danych A

Węzeł 2 przetwarza część danych B

Węzeł 3 przetwarza część danych C

Po zakończeniu pracy wyniki mogą zostać połączone.

Rozdzielanie obliczeń

W pamięci rozproszonej można rozdzielać obliczenia między różne węzły.

Każdy węzeł wykonuje swoją część pracy, co pozwala lepiej wykorzystać zasoby.

To daje:

- możliwość równoległego wykonania zadań,
 - lepsze wykorzystanie wielu komputerów,
 - większą odporność na ograniczenia pojedynczej maszyny,
 - możliwość pracy na bardzo dużych problemach.
-

Rozproszenie danych

Dane mogą być przechowywane na różnych komputerach.

Jest to korzystne, gdy:

- dane są bardzo duże,
 - dostępna pamięć jednej maszyny jest niewystarczająca,
 - chcemy przetwarzać dane blisko miejsca ich przechowywania,
 - różne części systemu odpowiadają za różne fragmenty danych.
-

Przesyłanie komunikatów

W modelu pamięci rozproszonej procesy zwykle komunikują się przez wiadomości.

Najczęściej używa się do tego MPI:

```
MPI_Send(...);
```

```
MPI_Recv(...);
```

Oznacza to, że programista często musi jawnie określić, jakie dane mają zostać wysłane i odebrane.

Synchronizacja między węzłami

Synchronizacja nadal może być potrzebna.

Przykładowo:

- węzły muszą poczekać na zakończenie pewnego etapu obliczeń,
- jeden proces musi zebrać wyniki od innych,
- dane muszą zostać wymienione przed następną fazą programu.

Dlatego pamięć rozproszona nie usuwa problemu synchronizacji, tylko zmienia sposób, w jaki ją realizujemy.

Najważniejsza intuicja

Pamięć rozproszona pozwala budować bardzo duże systemy z wielu komputerów, dzielić między nie dane i obliczenia, ale wymaga jawnej komunikacji oraz często synchronizacji między węzłami.

Pytanie 20: Interkomunikatory w MPI służą do:

Poprawne odpowiedzi:

- **Komunikacji między niezależnymi grupami procesów MPI**
- **Obsługi dynamicznego tworzenia nowych procesów MPI_Comm_spawn**

Niepoprawne odpowiedzi:

- **Przenoszenia danych tylko w ramach jednego procesu** — MPI służy do komunikacji między procesami, nie tylko wewnątrz jednego procesu.
- **Automatycznej synchronizacji pamięci współdzielonej pomiędzy procesami** — to nie jest zadanie interkomunikatorów.
- **Synchronizacji wątków w pamięci współdzielonej** — to dotyczy np. OpenMP, mutexów, semaforów, a nie interkomunikatorów MPI.
- **Komunikacji wewnątrz tej samej grupy procesów** — do tego służy **intrakomunikator**, np. MPI_COMM_WORLD.

Pojęcia do notatki

Komunikator w MPI

Komunikator w MPI określa grupę procesów, które mogą się ze sobą komunikować.

Najbardziej znany komunikator to:

MPI_COMM_WORLD

Obejmuje on wszystkie procesy uruchomione w danym programie MPI.

Intrakomunikator

Intrakomunikator służy do komunikacji **wewnątrz jednej grupy procesów**.

Przykład:

Grupa A:

P0, P1, P2, P3

Procesy z tej samej grupy mogą komunikować się ze sobą przez jeden intrakomunikator.

Typowy przykład:

MPI_COMM_WORLD

Interkomunikator

Interkomunikator służy do komunikacji **między dwiema różnymi grupami procesów MPI**.

Schemat:

Grupa A: P0, P1, P2



Grupa B: P3, P4, P5

Interkomunikator nie opisuje komunikacji wewnątrz jednej grupy, tylko połączenie między dwiema grupami.

MPI_Comm_spawn

Funkcja:

MPI_Comm_spawn(...)

służy do dynamicznego tworzenia nowych procesów MPI podczas działania programu.

Procesy rodzicielskie mogą utworzyć procesy potomne, a MPI tworzy między nimi **interkomunikator**, który pozwala tym grupom się komunikować.

Przykład intuicyjny:

Procesy rodzica uruchamiają nowe procesy potomne.

Rodzic ↔ potomkowie komunikują się przez interkomunikator.

Interkomunikator a pamięć współdzielona

Interkomunikator nie synchronizuje automatycznie pamięci współdzielonej.

MPI działa głównie w modelu **przesyłania komunikatów**, więc dane są przekazywane jawnie, np. przez:

MPI_Send(...)

MPI_Recv(...)

Najważniejsza intuicja

Interkomunikator łączy dwie oddzielne grupy procesów MPI, na przykład procesy rodzicielskie i procesy utworzone dynamicznie przez MPI_Comm_spawn.

Pytanie 21: Czym różni się Proof-of-Work od Proof-of-Stake w systemach blockchain?

Poprawne odpowiedzi:

- PoW wymaga intensywnych obliczeń, natomiast PoS opiera się na stakowaniu kryptowaluty
- PoS jest bardziej energooszczędny niż PoW
- PoW wykorzystuje proces kopania kryptowalut, a PoS nie wymaga takiej mocy obliczeniowej

Niepoprawne odpowiedzi:

- PoW bazuje na wyborze walidatora na podstawie ilości posiadanej kryptowaluty — to cecha PoS, nie PoW.
- PoS zwiększa decentralizację systemu blockchain w większym stopniu niż PoW — nie jest to ogólna reguła; zależy od konkretnej sieci.
- PoS wymaga użycia algorytmu SHA-256 — nie, SHA-256 jest znany głównie z Bitcoina i jego PoW, ale nie jest wymaganiem dla PoS.

Pojęcia do notatki

Blockchain

Blockchain to rozproszony rejestr danych, w którym informacje są zapisywane w blokach połączonych ze sobą kryptograficznie.

Każdy blok zawiera zwykle:

- dane transakcji,
- znacznik czasu,
- hash poprzedniego bloku,
- dodatkowe dane potrzebne do walidacji.

Celem blockchaina jest utrzymanie wspólnej, trudnej do sfałszowania historii transakcji bez jednej centralnej instytucji.

Mechanizm konsensusu

Mechanizm konsensusu to sposób, w jaki uczestnicy sieci blockchain zgadzają się, który blok jest poprawny i może zostać dodany do łańcucha.

Dwa popularne mechanizmy to:

Proof-of-Work → praca obliczeniowa
Proof-of-Stake → stakowanie kryptowaluty

Proof-of-Work, czyli PoW

Proof-of-Work polega na tym, że uczestnicy sieci, czyli górnicy, rozwiązują trudne zadania obliczeniowe.

Najczęściej chodzi o znalezienie takiego wyniku funkcji hashującej, który spełnia określony warunek.

Proces ten nazywa się **kopaniem**, czyli **miningiem**.

Cechy PoW:

- wymaga dużej mocy obliczeniowej,
 - zużywa dużo energii,
 - bezpieczeństwo opiera się na koszcie wykonania obliczeń,
 - przykład: Bitcoin.
-

Kopanie kryptowalut

Kopanie kryptowalut to proces tworzenia nowych bloków w systemie Proof-of-Work.

Górnicy wykonują wiele prób obliczeniowych, aby znaleźć poprawny blok.

Ten, kto pierwszy znajdzie poprawne rozwiązanie, może dodać blok do blockchaina i zwykle otrzymuje nagrodę.

Proof-of-Stake, czyli PoS

Proof-of-Stake polega na tym, że walidatorzy bloków są wybierani na podstawie posiadanej i zablokowanej kryptowaluty, czyli **stake'u**.

Zamiast inwestować w moc obliczeniową, uczestnik „stawia” część swoich środków jako zabezpieczenie.

Cechy PoS:

- nie wymaga energochłonnego kopania,
 - opiera się na stakowaniu kryptowaluty,
 - jest zwykle bardziej energooszczędny niż PoW,
 - walidator może stracić część stake'u za nieuczciwe działanie.
-

Staking

Staking oznacza zablokowanie określonej ilości kryptowaluty, aby uczestniczyć w walidowaniu bloków.

Im większy stake, tym większa szansa na wybór jako walidator, choć konkretne zasady zależą od danej sieci blockchain.

Energooszczędność PoS

PoS jest zwykle bardziej energooszczędny niż PoW, ponieważ nie wymaga masowego wykonywania obliczeń przez górników.

W PoW wiele urządzeń konkuruje, wykonując ogromną liczbę prób.

W PoS wybór walidatora odbywa się bez tak kosztownego obliczeniowo procesu.

SHA-256

SHA-256 to funkcja hashująca używana między innymi w Bitcoinie.

Nie jest jednak obowiązkową częścią każdego systemu PoW ani tym bardziej PoS.

Dlatego stwierdzenie, że PoS wymaga SHA-256, jest błędne.

Najważniejsza intuicja

PoW zabezpiecza sieć przez kosztowną pracę obliczeniową, a PoS przez ekonomiczny zastaw kryptowaluty; dlatego PoS zwykle zużywa znacznie mniej energii niż PoW.

Pytanie 22: Które z poniższych etapów są typowe dla przetwarzania potokowego?

Poprawne odpowiedzi:

- **Pobranie instrukcji**
- **Dekodowanie instrukcji**
- **Zapis instrukcji** — najpewniej chodzi o etap **zapisu wyniku**, czyli *write-back*

Niepoprawne odpowiedzi:

- **Przerywanie** — przerwania mogą zakłócić potok, ale nie są typowym etapem potoku.
 - **Komunikacja między procesami** — to temat systemów operacyjnych i programowania równoległego, nie etap potoku procesora.
 - **Synchronizacja pamięci podręcznej** — ważna w systemach wielordzeniowych, ale nie jest klasycznym etapem potoku instrukcji.
-

Pojęcia do notatki

Przetwarzanie potokowe

Przetwarzanie potokowe, czyli **pipelining**, to technika wykonywania instrukcji procesora etapami.

Zamiast wykonywać jedną instrukcję od początku do końca, procesor dzieli jej wykonanie na fazy i jednocześnie przetwarza różne instrukcje na różnych etapach.

Przykład intuicyjny:

Instrukcja 1: pobieranie → dekodowanie → wykonanie → zapis

Instrukcja 2: pobieranie → dekodowanie → wykonanie → zapis

Instrukcja 3: pobieranie → dekodowanie → wykonanie → zapis

Dzięki temu procesor może zwiększyć przepustowość wykonywania instrukcji.

Pobranie instrukcji

Pobranie instrukcji, czyli **instruction fetch**, to etap, w którym procesor pobiera kolejną instrukcję z pamięci.

Procesor sprawdza adres instrukcji, zwykle zapisany w liczniku programu, czyli **program counter**.

Dekodowanie instrukcji

Dekodowanie instrukcji, czyli **instruction decode**, polega na rozpoznaniu, co dana instrukcja ma zrobić.

Procesor ustala na przykład:

- jaka operacja ma zostać wykonana,
 - jakie rejestry są używane,
 - czy potrzebny jest dostęp do pamięci,
 - jakie argumenty są wymagane.
-

Wykonanie instrukcji

Chociaż tej opcji nie ma na liście, jest to klasyczny etap potoku.

Wykonanie instrukcji polega na przeprowadzeniu właściwej operacji, na przykład dodawania, porównania albo obliczenia adresu.

Zapis wyniku

W klasycznym potoku występuje etap **write-back**, czyli **zapis wyniku**.

Wynik operacji jest zapisywany na przykład do rejestru procesora.

Uwaga: w pytaniu jest sformułowanie „**zapis instrukcji**”, ale standardowo mówi się raczej „**zapis wyniku**”, nie „zapis instrukcji”.

Najważniejsza intuicja

Potok procesora działa jak taśma produkcyjna: jedna instrukcja jest pobierana, druga dekodowana, trzecia wykonywana, a czwarta zapisuje wynik.

Pytanie 23: Wykonanie poza kolejnością, czyli **Out-of-Order Execution**

Poprawne odpowiedzi:

- Oznacza, że procesor może dynamicznie zmieniać kolejność wykonywania instrukcji, aby zoptymalizować wydajność
- Może prowadzić do problemów związanych z zależnościami między instrukcjami
W pytaniu jest literówka: „infrastrukturami” — najpewniej chodziło o **instrukcjami**.

Niepoprawne odpowiedzi:

- Znaczy, że kolejność wykonywania instrukcji zawsze musi być zgodna z kolejnością w kodzie — to opis wykonania w kolejności, czyli *in-order execution*.
- Oznacza, że instrukcje są wykonywane zgodnie z priorytetem nadanym przez programistę — procesor sam decyduje o kolejności na podstawie dostępności danych i zasobów.
- Znaczy, że każda instrukcja musi być wykonywana na oddzielnym rdzeniu — out-of-order dotyczy wykonywania instrukcji wewnątrz procesora/rdzenia, nie koniecznie wielu rdzeni.

Pojęcia do notatki

Out-of-Order Execution

Out-of-Order Execution to technika stosowana w procesorach, która pozwala wykonywać instrukcje w innej kolejności niż ta, w której występują w kodzie programu.

Procesor może zmienić kolejność wykonania instrukcji, jeśli nie łamie to zależności między nimi.

Celem jest lepsze wykorzystanie jednostek wykonawczych procesora.

Wykonanie w kolejności — In-Order Execution

W modelu **in-order** instrukcje są wykonywane dokładnie w takiej kolejności, w jakiej występują w programie.

Przykład:

Instrukcja 1
Instrukcja 2
Instrukcja 3
Instrukcja 4

Jeśli instrukcja 2 czeka na dane z pamięci, procesor może musieć czekać, nawet jeśli instrukcja 3 mogłaby zostać wykonana wcześniej.

Wykonanie poza kolejnością

W modelu **out-of-order** procesor może wykonać późniejszą instrukcję wcześniej, jeśli jest ona gotowa do wykonania.

Przykład:

Instrukcja 1: długo czeka na dane z pamięci

Instrukcja 2: zależy od instrukcji 1

Instrukcja 3: niezależna od instrukcji 1 i 2

Procesor może wykonać instrukcję 3 wcześniej, zamiast beczynnie czekać.

Zależności między instrukcjami

Procesor nie może dowolnie przestawiać wszystkich instrukcji.

Musi uważać na zależności, na przykład:

$a = b + c;$

$d = a * 2;$

Dруга instrukcja zależy od wyniku pierwszej, więc nie może zostać wykonana wcześniej.

Optymalizacja wydajności

Out-of-order execution poprawia wydajność, ponieważ procesor może:

- wykonywać gotowe instrukcje wcześniej,
 - ukrywać opóźnienia dostępu do pamięci,
 - lepiej wykorzystywać jednostki arytmetyczne,
 - zmniejszać czas beczynności rdzenia.
-

Kolejność widoczna dla programu

Mimo że procesor może wykonywać instrukcje poza kolejnością wewnętrznie, wynik programu powinien wyglądać tak, jakby instrukcje zostały wykonane zgodnie z kolejnością programu.

To znaczy, że procesor musi zachować poprawność semantyczną programu.

Najważniejsza intuicja

Out-of-order execution pozwala procesorowi wykonywać najpierw te instrukcje, które są już gotowe, zamiast czekać na instrukcje opóźnione przez dane lub pamięć.

Test 2

Pytanie 1: Jakiego mechanizmu używamy w POSIX Threads do zabezpieczania dostępu do sekcji krytycznej?

Poprawna odpowiedź:

b) pthread_mutex_lock i pthread_mutex_unlock

Pojęcia do notatki

POSIX Threads / Pthreads

POSIX Threads, czyli **Pthreads**, to standard programowania wielowątkowego w językach C/C++ na systemach zgodnych z POSIX, np. Linux.

Pozwala tworzyć i synchronizować wątki za pomocą funkcji zaczynających się od pthread_.

Sekcja krytyczna

Sekcja krytyczna to fragment kodu, w którym wątek korzysta ze współdzielonych danych.

Przykład:

```
licznik++;
```

Jeśli kilka wątków jednocześnie modyfikuje licznik, może dojść do błędów współbieżności.

Mutex

Mutex to mechanizm wzajemnego wykluczania.

Pozwala zagwarantować, że w danym momencie tylko jeden wątek wykonuje chroniony fragment kodu.

W Pthreads używa się:

```
pthread_mutex_lock(&mutex);  
// sekcja krytyczna  
pthread_mutex_unlock(&mutex);
```

pthread_mutex_lock blokuje mutex, a pthread_mutex_unlock go odblokowuje.

Dlaczego pozostałe odpowiedzi są błędne?

#pragma omp critical należy do **OpenMP**, nie do POSIX Threads.

MPI_Send i MPI_Recv służą do komunikacji między procesami w **MPI**, a nie do ochrony sekcji krytycznej w Pthreads.

atomic może chronić pojedyncze operacje atomowe, ale klasycznym mechanizmem sekcji krytycznej w Pthreads jest **mutex**.

Najważniejsza intuicja

W Pthreads sekcję krytyczną zabezpieczamy muteksem: przed wejściem robimy `pthread_mutex_lock`, a po wyjściu `pthread_mutex_unlock`.

Pytanie 2: Co oznacza `#pragma omp parallel for` w kontekście OpenMP?

Poprawna odpowiedź:

a. Uruchamia pętlę równoległą, dzieląc jej iteracje na dostępne wątki

Pojęcia do notatki

OpenMP

OpenMP to technologia do programowania równoległego w modelu pamięci wspólnej.

Pozwala łatwo zrównoleglić fragmenty programu za pomocą dyrektyw kompilatora, np.:

`#pragma omp parallel`

albo:

`#pragma omp parallel for`

`#pragma omp parallel for`

Dyrektywa:

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    oblicz(i);
}
```

oznacza, że iteracje pętli `for` zostaną podzielone między dostępne wątki.

Na przykład dla 8 iteracji i 4 wątków może wyglądać to tak:

Wątek 0: iteracje 0, 1

Wątek 1: iteracje 2, 3

Wątek 2: iteracje 4, 5

Wątek 3: iteracje 6, 7

Każdy wątek wykonuje część iteracji, dzięki czemu cała pętla może zakończyć się szybciej.

Różnica między `parallel` a `parallel for`

Sama dyrektywa:

`#pragma omp parallel`

tworzy równoległy obszar kodu. Każdy wątek wykonuje ten sam blok.

Natomiast:

`#pragma omp parallel for`

tworzy zespół wątków i dzieli iteracje pętli między te wątki.

Czyli:

`parallel` → wiele wątków wykonuje blok kodu

`parallel for` → wiele wątków dzieli między siebie iteracje pętli

Czy tworzony jest osobny wątek dla każdej iteracji?

Nie. `#pragma omp parallel for` **nie tworzy osobnego wątku dla każdej iteracji**.

Tworzona jest grupa wątków, a iteracje są między nie rozdzielane. Jeden wątek może wykonać wiele iteracji.

Dlaczego pozostałe odpowiedzi są błędne?

b. Uruchamia pętlę sekwencyjnie w równoległym obszarze — nie, celem `parallel for` jest równoległe wykonanie iteracji.

c. Uruchamia nowy wątek dla każdej iteracji pętli — nie, liczba wątków zwykle jest ograniczona, a iteracje są między nie dzielone.

d. Nie jest związane z pętlą, tworzy równoległy blok kodu — to bardziej opisuje `#pragma omp parallel`, a nie `parallel for`.

Najważniejsza intuicja

`#pragma omp parallel for` mówi OpenMP: **podziel iteracje tej pętli między wiele wątków i wykonuj je równoległe**.

Pytanie 3: Jak w MPI można zrealizować komunikację jednego do wielu?

Poprawna odpowiedź:

a. MPI_Bcast

Pojęcia do notatki

Komunikacja jeden-do-wielu

Komunikacja jeden-do-wielu oznacza, że jeden proces wysyła tę samą informację do wielu innych procesów.

Schemat:

Proces 0 → Proces 1
Proces 0 → Proces 2
Proces 0 → Proces 3
Proces 0 → Proces 4

W MPI typową funkcją do tego jest `MPI_Bcast`.

MPI_Bcast

`MPI_Bcast`, czyli **broadcast**, służy do rozgłoszenia danych z jednego procesu do wszystkich procesów w danym komunikatorze.

Przykład:

```
MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Oznacza to, że proces o numerze 0 wysyła wartość zmiennej `x` do wszystkich pozostałych procesów w `MPI_COMM_WORLD`.

Dlaczego pozostałe odpowiedzi są błędne?

MPI_Gather działa odwrotnie: zbiera dane od wielu procesów do jednego procesu.

Wielu → jeden

MPI_Barrier służy do synchronizacji procesów, a nie do przesyłania danych.

MPI_Allgather zbiera dane od wszystkich procesów i rozsyła wynik do wszystkich.

Wielu → wielu

Najważniejsza intuicja

MPI_Bcast to rozgłoszenie: jeden proces ma dane, a MPI rozsyła je do wszystkich pozostałych procesów.

Pytanie 4: Co oznacza zależność WAW, czyli **Write After Write**?

Poprawna odpowiedź:

a. Pierwszy zapis musi zostać wykonany przed drugim zapisem

Pojęcia do notatki

WAW — Write After Write

WAW, czyli **Write After Write**, oznacza zależność między dwoma zapisami do tego samego miejsca, na przykład do tej samej zmiennej lub rejestru.

Chodzi o to, że zapisy muszą zachować właściwą kolejność, żeby końcowa wartość była poprawna.

Przykład:

```
x = 5; // pierwszy zapis  
x = 10; // drugi zapis
```

Po wykonaniu programu końcowa wartość x powinna wynosić 10.

Jeśli procesor lub kompilator zmieniłby kolejność zapisów, mogłoby powstać:

```
x = 10;  
x = 5;
```

Wtedy końcowa wartość byłaby błędna, bo x wyniosłoby 5.

Zależność wyjściowa

WAW nazywa się też **zależnością wyjściową**, ponieważ dwie instrukcje zapisują do tego samego miejsca docelowego.

Nie chodzi tu o odczyt danych, tylko o kolejność zapisów.

Znaczenie w wykonaniu poza kolejnością

WAW jest ważne przy:

- wykonywaniu poza kolejnością, czyli **out-of-order execution**,
- optymalizacjach kompilatora,
- potokowym wykonaniu instrukcji,
- równoległym wykonywaniu instrukcji.

Procesor może zmieniać kolejność instrukcji tylko wtedy, gdy nie zmieni to wyniku programu.

Porównanie z innymi zależnościami

RAW — Read After Write

Odczyt po zapisie. Instrukcja musi najpierw zapisać wartość, zanim kolejna ją odczyta.

```
x = 5;  
y = x;
```

WAR — Write After Read

Zapis po odczycie. Najpierw trzeba odczytać starą wartość, zanim zostanie nadpisana.

```
y = x;  
x = 5;
```

WAW — Write After Write

Zapis po zapisie. Kolejność zapisów musi zostać zachowana.

```
x = 5;  
x = 10;
```

Najważniejsza intuicja

WAW oznacza, że jeśli dwie instrukcje zapisują do tego samego miejsca, to drugi zapis nie może „wyprowadzić” pierwszego, bo zmieniłoby to końcowy wynik.

Pytanie 5: Które stwierdzenia dotyczące POSIX Threads i zmiennych warunkowych są prawdziwe?

Poprawne odpowiedzi:

a. `pthread_cond_wait()` powoduje, że wątek czeka na zmiennej warunkowej, a w tym czasie zwalnia blokadę, którą wątek może posiadać

b. `pthread_cond_signal()` budzi jeden wątek czekający na zmiennej warunkowej

Dlaczego pozostałe są błędne?

c. `pthread_cond_broadcast()` budzi tylko jeden wątek czekający na zmiennej warunkowej — błędne, bo `pthread_cond_broadcast()` budzi **wszystkie** wątki czekające na danej zmiennej warunkowej.

d. `pthread_mutex_lock()` może być użyte do zablokowania zmiennej warunkowej — błędne. `pthread_mutex_lock()` blokuje **mutex**, a nie zmienną warunkową. Mutex jest zwykle używany razem ze zmienną warunkową, ale nie blokuje jej bezpośrednio.

e. `pthread_cond_t` jest używany do inicjalizacji mutexów — błędne. `pthread_cond_t` to typ zmiennej warunkowej. Do mutexów używa się `pthread_mutex_t`.

Pojęcia do notatki

Zmienna warunkowa

Zmienna warunkowa w Pthreads służy do usypiania wątku do momentu, aż zostanie spełniony określony warunek.

Przykład sytuacji:

Wątek czeka, aż bufor nie będzie pusty.

Wątek czeka, aż pojawi się nowe zadanie.

Wątek czeka, aż inny wątek zakończy etap pracy.

Zmienna warunkowa zwykle działa razem z **mutexem**.

`pthread_cond_wait()`

Funkcja:

```
pthread_cond_wait(&cond, &mutex);
```

powoduje, że wątek zaczyna czekać na zmiennej warunkowej `cond`.

Bardzo ważne: podczas czekania funkcja **zwalnia mutex**, aby inne wątki mogły wejść do sekcji krytycznej i zmienić warunek.

Po przebudzeniu wątek ponownie przejmuje mutex.

Schemat:

1. Wątek ma mutex.
 2. Wywołuje `pthread_cond_wait()`.
 3. Mutex zostaje zwolniony.
 4. Wątek śpi.
 5. Inny wątek sygnalizuje warunek.
 6. Czekający wątek budzi się i ponownie blokuje mutex.
-

pthread_cond_signal()

Funkcja:

```
pthread_cond_signal(&cond);
```

budzi **jeden** wątek czekający na danej zmiennej warunkowej.

Jeśli nie ma żadnego czekającego wątku, sygnał zwykle przepada.

pthread_cond_broadcast()

Funkcja:

```
pthread_cond_broadcast(&cond);
```

budzi **wszystkie** wątki czekające na danej zmiennej warunkowej.

Jest używana wtedy, gdy zmiana warunku może dotyczyć wielu wątków naraz.

Mutex a zmienna warunkowa

Mutex chroni dane współdzielone, a zmienna warunkowa pozwala czekać na zmianę stanu tych danych.

Typowy schemat:

```
pthread_mutex_lock(&mutex);
```

```
while (!warunek) {  
    pthread_cond_wait(&cond, &mutex);  
}
```

```
// sekcja wykonywana, gdy warunek jest spełniony
```

```
pthread_mutex_unlock(&mutex);
```

Używa się `while`, a nie `if`, ponieważ wątek po przebudzeniu powinien ponownie sprawdzić, czy warunek rzeczywiście jest spełniony.

Najważniejsza intuicja

Zmienna warunkowa pozwala wątkowi spać do czasu spełnienia warunku, a `pthread_cond_wait()` podczas czekania zwalnia mutex, żeby inne wątki mogły ten warunek zmienić.

Pytanie 6: Jakie jest główne zadanie dyrektywy `#pragma omp critical` w OpenMP?

Poprawna odpowiedź:

c. Zabezpieczenie sekcji krytycznej

Pojęcia do notatki

`#pragma omp critical`

Dyrektywa:

```
#pragma omp critical
{
    // sekcja krytyczna
}
```

oznacza, że dany fragment kodu może być wykonywany tylko przez **jeden wątek naraz**.

Pozostałe wątki muszą poczekać, aż aktualny wątek opuści sekcję krytyczną.

Sekcja krytyczna

Sekcja krytyczna to fragment programu, w którym wiele wątków może odwoływać się do tych samych danych współdzielonych.

Przykład:

```
#pragma omp critical
{
    suma += wynik_lokalny;
}
```

Bez zabezpieczenia kilka wątków mogłoby jednocześnie zmieniać zmienną `suma`, powodując błędny wynik.

Synchronizacja wątków

`#pragma omp critical` jest jedną z form synchronizacji, ale jej główne zadanie to **ochrona sekcji krytycznej**.

Dlatego odpowiedź **b. Synchronizacja wątków** jest częściowo ogólna, ale mniej precyzyjna niż **c**.

Dlaczego pozostałe odpowiedzi są błędne?

a. Utworzenie nowego wątku — wątki tworzy np. `#pragma omp parallel`.

b. Synchronizacja wątków — to ogólne określenie; `critical` synchronizuje dostęp, ale konkretnie zabezpiecza sekcję krytyczną.

d. Uruchomienie pętli w równoległym bloku kodu — to robi np. `#pragma omp parallel for`.

Najważniejsza intuicja

`#pragma omp critical` działa jak bramka: do chronionego fragmentu kodu może wejść tylko jeden wątek naraz.

Pytanie 7: Co jest celem funkcji `pthread_join`?

Poprawna odpowiedź:

b. Czekanie na zakończenie wątku i pobranie jego wartości zwracanej.

Pojęcia do notatki

`pthread_join`

Funkcja:

```
pthread_join(thread, &retval);
```

służy do tego, aby jeden wątek poczekał na zakończenie innego wątku.

Najczęściej używa jej wątek główny, aby poczekać, aż utworzone wcześniej wątki zakończą pracę.

Czekanie na zakończenie wątku

Jeśli wątek główny utworzy dodatkowy wątek przez:

```
pthread_create(&thread, NULL, funkcja, NULL);
```

to może potem poczekać na jego zakończenie:

```
pthread_join(thread, NULL);
```

Dzięki temu program nie kończy się zanim wątki wykonają swoją pracę.

Wartość zwracana przez wątek

Wątek może zakończyć się i zwrócić pewną wartość, którą można odebrać przez `pthread_join`.

Przykład ogólny:

```
void *wynik;
```

```
pthread_join(thread, &wynik);
```

Jeśli nie interesuje nas wartość zwracana, można podać NULL.

Dlaczego pozostałe odpowiedzi są błędne?

a. Tworzenie nowego wątku — do tego służy `pthread_create`.

c. Zakończenie wątku — wątek może zakończyć się sam przez `return` z funkcji wątku albo przez `pthread_exit`.

d. Zainicjowanie mutexa — do tego służy `pthread_mutex_init`.

Najważniejsza intuicja

`pthread_join` oznacza: **poczekaj, aż wskazany wątek skończy działanie, a opcjonalnie odbierz wynik jego pracy.**

Pytanie 8: W MPI, co robi funkcja `MPI_Reduce`?

Poprawna odpowiedź:

a. Zbiera dane od wszystkich procesów i wykonuje na nich operację redukującą.

Pojęcia do notatki

MPI_Reduce

`MPI_Reduce` to funkcja komunikacji zbiorowej w MPI.

Służy do zebrania danych od wielu procesów i wykonania na nich jednej operacji, na przykład:

- sumowania,
- wyznaczenia maksimum,
- wyznaczenia minimum,
- mnożenia,
- operacji logicznej.

Wynik trafia do jednego wybranego procesu, nazywanego **root**.

Redukcja

Redukcja oznacza połączenie wielu wartości w jedną wartość wynikową.

Przykład:

Proces 0 ma: 2

Proces 1 ma: 5

Proces 2 ma: 3

Proces 3 ma: 10

Po wykonaniu redukcji z operacją sumowania:

$$2 + 5 + 3 + 10 = 20$$

Wynik 20 trafia do procesu root.

Przykład użycia

```
int lokalna_suma;  
int globalna_suma;
```

```
MPI_Reduce(  
    &lokalna_suma,  
    &globalna_suma,  
    1,  
    MPI_INT,  
    MPI_SUM,  
    0,  
    MPI_COMM_WORLD  
);
```

Tutaj każdy proces ma swoją lokalna_suma, a proces 0 otrzymuje sumę wszystkich wartości w globalna_suma.

Dlaczego pozostałe odpowiedzi są błędne?

- b. Wysła dane od jednego procesu do wszystkich innych — to opisuje MPI_Bcast.
 - c. Synchronizuje wszystkie procesy — do tego służy MPI_Barrier.
 - d. Rozsyła dane od jednego procesu do wielu — to również opisuje MPI_Bcast, czyli broadcast.
-

Najważniejsza intuicja

MPI_Reduce działa jak „zbierz wartości od wszystkich, wykonaj na nich operację, a wynik zapisz u jednego procesu”.

Pytanie 9: Jak można zapobiec zależnościom WAR, czyli **Write After Read**, w pętli?

Poprawna odpowiedź:

- b. Zastosowanie zmiennych lokalnych dla wątków.
-

Pojęcia do notatki

WAR — Write After Read

WAR, czyli **Write After Read**, oznacza zależność, w której najpierw musi nastąpić **odczyt** starej wartości, a dopiero potem można wykonać **zapis** do tego samego miejsca.

Przykład:

```
y = x; // najpierw odczyt x
x = 10; // potem zapis do x
```

Jeśli zapis `x = 10` wykonałby się za wcześniej, to instrukcja `y = x` odczytałaby już nową wartość, a nie starą.

WAR w pętli

W pętli zależność WAR może pojawić się wtedy, gdy jedna iteracja odczytuje zmienną, a inna iteracja później ją nadpisuje.

Przykład intuicyjny:

```
for (int i = 1; i < n; i++) {
    a[i - 1] = a[i];
}
```

Tutaj jedna iteracja zapisuje do elementu tablicy, który inna iteracja może chcieć wcześniej odczytać.

Zmienne lokalne dla wątków

Jednym ze sposobów unikania zależności WAR jest użycie **zmiennych lokalnych**, czyli prywatnych kopii danych dla wątków.

Zamiast wiele wątków odczytywało i zapisywało tę samą zmienną, każdy wątek pracuje na swojej kopii.

Przykład:

```
#pragma omp parallel private(tmp)
{
    int tmp;
    tmp = oblicz_lokalnie();
}
```

Dzięki temu wątki nie nadpisują sobie danych, które inne wątki muszą jeszcze odczytać.

Prywatne dane w OpenMP

W OpenMP można użyć klauzuli:

```
private(zmienna)
```

albo zadeklarować zmienną wewnątrz bloku równoległego:

```
#pragma omp parallel
{
    int lokalna; // osobna zmienna dla każdego wątku
}
```

To pomaga uniknąć konfliktów dostępu do wspólnych zmiennych.

Dlaczego pozostałe odpowiedzi są błędne?

a. Zastosowanie operacji atomowych — operacje atomowe chronią pojedyncze odczyty/zapisy lub modyfikacje, ale nie rozwiązują ogólnie problemu zależności WAR w pętli.

c. Zastosowanie mechanizmu mutex — mutex może wymusić kolejność i zabezpieczyć dostęp, ale zwykle serializuje wykonanie i nie jest typowym sposobem usuwania zależności WAR w pętli.

d. Zastosowanie metody MPI_Allreduce — to operacja zbiorowa MPI do redukcji danych między procesami, nie mechanizm usuwania zależności WAR.

Najważniejsza intuicja

WAR oznacza konflikt: ktoś musi jeszcze odczytać starą wartość, a ktoś inny chce ją już nadpisać; użycie zmiennych lokalnych/prywatnych kopii pomaga uniknąć takiego nadpisywania.

Pytanie 10: Jakie są zalety i wady użycia `#pragma omp parallel for schedule(dynamic, X)` w OpenMP?

Poprawne odpowiedzi:

a. Zalety to łatwość implementacji i dynamiczne przypisanie iteracji; wady to potencjalne narzuty czasowe.

c. Zalety to równomierne rozłożenie obciążenia; wady to narzuty czasowe.

Pojęcia do notatki

`schedule(dynamic, X)` w OpenMP

Klauzula:

```
#pragma omp parallel for schedule(dynamic, X)
```

oznacza, że iteracje pętli są przydzielane wątkom **dynamicznie**, w porcjach o rozmiarze `X`.

Czyli wątek nie dostaje całej swojej pracy od razu. Zamiast tego pobiera kolejną paczkę iteracji, gdy skończy poprzednią.

Dynamiczne przypisywanie iteracji

W `schedule(dynamic, X)` OpenMP dzieli iteracje na porcje, czyli **chunki**.

Przykład:

```
#pragma omp parallel for schedule(dynamic, 2)
for (int i = 0; i < 10; i++) {
    oblicz(i);
}
```

Dla $X = 2$ iteracje są rozdawane w paczkach po 2:

paczka 1: iteracje 0, 1

paczka 2: iteracje 2, 3

paczka 3: iteracje 4, 5

...

Wątek, który skończy szybciej, bierze następną dostępną paczkę.

Zaleta: lepsze równoważenie obciążenia

dynamic jest przydatne, gdy iteracje pętli mają różny czas wykonania.

Przykład:

iteracja 0: krótka

iteracja 1: długa

iteracja 2: krótka

iteracja 3: bardzo długa

Przy statycznym podziale jeden wątek mógłby dostać dużo cięższej pracy niż pozostałe.

Przy dynamicznym podziale szybciej kończące wątki pobierają kolejne iteracje, więc obciążenie rozkłada się równomierniej.

Zaleta: łatwość implementacji

Programista nie musi samodzielnie pisać mechanizmu kolejki zadań.

Wystarczy dopisać:

```
schedule(dynamic, X)
```

i OpenMP zajmuje się przydzielaniem iteracji w czasie działania programu.

Wada: narzut czasowy

Dynamiczne przydzielanie iteracji ma koszt.

OpenMP musi zarządzać tym, które porcje pracy zostały już wykonane i które można przydzielić kolejnym wątkom.

Narzut może wynikać z:

- częstszego przydzielania pracy,
- synchronizacji przy pobieraniu kolejnych paczek,
- zarządzania kolejką/chunkami iteracji.

Im mniejsze X , tym lepsze balansowanie, ale większy narzut.

Im większe X , tym mniejszy narzut, ale słabsze równoważenie obciążenia.

Dlaczego pozostałe odpowiedzi są błędne?

b. Zalety to brak narzutów czasowych i pełna kontrola nad iteracjami; wady to trudność implementacji — błędne, bo dynamic ma narzut czasowy, a implementacja jest raczej prosta.

d. Zalety to szybkość i brak narzutów czasowych; wady to trudność implementacji — błędne, bo dynamiczne planowanie nie jest pozbawione narzutów.

Najważniejsza intuicja

schedule(dynamic, X) pomaga, gdy iteracje mają różny koszt, bo wątki dobierają sobie kolejne porcje pracy, ale płacimy za to dodatkowym narzutem zarządzania.

Pytanie 11: Co oznacza `MPI_Barrier(MPI_COMM_WORLD)`?

Poprawna odpowiedź:

b. Synchronizuje wszystkie procesy w MPI.

Pojęcia do notatki

MPI_Barrier

MPI_Barrier to funkcja synchronizująca procesy w MPI.

Wywołanie:

```
MPI_Barrier(MPI_COMM_WORLD);
```

oznacza, że każdy proces należący do komunikatora MPI_COMM_WORLD musi dojść do tej bariery.

Proces, który dojdzie wcześniej, czeka na pozostałe.

Bariera synchronizacyjna

Bariera działa jak punkt kontrolny.

Schemat:

Proces 0 dochodzi do bariery → czeka

Proces 1 dochodzi do bariery → czeka

Proces 2 dochodzi do bariery → czeka

Proces 3 dochodzi do bariery → wszyscy mogą iść dalej

Dopiero gdy wszystkie procesy osiągną barierę, każdy z nich może kontynuować wykonanie programu.

MPI_COMM_WORLD

MPI_COMM_WORLD to domyślny komunikator obejmujący wszystkie procesy uruchomione w programie MPI.

Czyli:

```
MPI_Barrier(MPI_COMM_WORLD);
```

synchronizuje wszystkie procesy programu.

Dlaczego pozostałe odpowiedzi są błędne?

a. Zakończa komunikację w MPI — nie, MPI_Barrier tylko synchronizuje procesy.

c. Inicjuje MPI — do inicjalizacji MPI służy:

```
MPI_Init(...);
```

d. Zakończa MPI — do zakończenia pracy z MPI służy:

```
MPI_Finalize();
```

Najważniejsza intuicja

MPI_Barrier(MPI_COMM_WORLD) oznacza: wszystkie procesy mają poczekać w tym miejscu, aż każdy z nich tam dotrze.

Pytanie 12: W jaki sposób można zainicjować mutex w Pthreads?

Poprawne odpowiedzi:

a. pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

e. pthread_mutex_init(&mutex); — z doprecyzowaniem: w praktyce poprawne wywołanie ma zwykle postać:

```
pthread_mutex_init(&mutex, NULL);
```

Pojęcia do notatki

Mutex w Pthreads

Mutex to mechanizm synchronizacji służący do ochrony sekcji krytycznej.

Deklaracja mutex:

```
pthread_mutex_t mutex;
```

Przed użyciem mutex musi zostać zainicjalizowany.

Inicjalizacja statyczna

Mutex można zainicjalizować statycznie:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

Ten sposób jest prosty i często używany dla mutexów globalnych lub statycznych.

Przykład:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&mutex);
// sekcja krytyczna
pthread_mutex_unlock(&mutex);
```

Inicjalizacja dynamiczna

Mutex można też zainicjalizować funkcją:

```
pthread_mutex_init(&mutex, NULL);
```

Drugi argument określa atrybuty mutex'a. Jeśli podamy NULL, używane są atrybuty domyślne.

Przykład:

```
pthread_mutex_t mutex;

pthread_mutex_init(&mutex, NULL);

pthread_mutex_lock(&mutex);
// sekcja krytyczna
pthread_mutex_unlock(&mutex);

pthread_mutex_destroy(&mutex);
```

Dlaczego pozostałe odpowiedzi są błędne?

b. `pthread_mutex_t mutex = PTHREAD_CREATE_INITIALIZER;` — taka stała nie służy do inicjalizacji mutex'a.

c. `#pragma omp critical` — to mechanizm OpenMP, nie Pthreads.

d. `MPI_Init(&mutex);` — `MPI_Init` inicjuje środowisko MPI, a nie mutex.

Najważniejsza intuicja

Mutex w Pthreads inicjalizujemy albo statycznie przez `PTHREAD_MUTEX_INITIALIZER`, albo dynamicznie przez `pthread_mutex_init(&mutex, NULL)`.

Pytanie 13: Jakie są główne różnice między przetwarzaniem równoległym a rozproszonym?

Poprawne odpowiedzi:

a. Równoległe wykonuje się na jednym komputerze, rozproszone na wielu maszynach.

c. Równoległe używa pamięci wspólnej, rozproszone pamięci rozproszonej.

d. W równoległym istnieje globalny zegar, w rozproszonym nie.

Niepoprawna odpowiedź:

b. Równoległe zawsze jest szybsze od rozproszonego — nie zawsze. Wydajność zależy od problemu, komunikacji, synchronizacji, liczby maszyn, sieci i narzutów.

Pojęcia do notatki

Przetwarzanie równoległe

Przetwarzanie równoległe polega na jednoczesnym wykonywaniu wielu części programu.

Najczęściej kojarzy się z jednym komputerem wielordzeniowym, gdzie wiele wątków działa w ramach jednej maszyny.

Przykład:

Jeden komputer

- └— rdzeń 1 wykonuje część A
- └— rdzeń 2 wykonuje część B
- └— rdzeń 3 wykonuje część C
- └— rdzeń 4 wykonuje część D

Typowe technologie:

OpenMP
Pthreads
CUDA

Przetwarzanie rozproszone

Przetwarzanie rozproszone polega na wykonywaniu obliczeń na wielu niezależnych komputerach połączonych siecią.

Każda maszyna może mieć własny procesor, pamięć, system operacyjny i lokalne zasoby.

Przykład:

Komputer 1 wykonuje część A
Komputer 2 wykonuje część B
Komputer 3 wykonuje część C
Komputer 4 wykonuje część D

Typowe technologie:

MPI
PVM
systemy gridowe
systemy chmurowe

Pamięć wspólna a pamięć rozproszona

W przetwarzaniu równoległym często używa się **pamięci wspólnej**.

Oznacza to, że wiele wątków ma dostęp do tych samych zmiennych i struktur danych.

Wątki → wspólna pamięć

W przetwarzaniu rozproszonym używa się zwykle **pamięci rozproszonej**.

Każdy komputer ma własną pamięć, a dane trzeba przesyłać przez sieć.

Proces 1: własna pamięć

Proces 2: własna pamięć

Proces 3: własna pamięć

Globalny zegar

W systemach rozproszonych zwykle **nie ma jednego globalnego zegara**, który idealnie synchronizuje wszystkie maszyny.

Każdy komputer ma własny zegar, który może działać minimalnie inaczej.

To powoduje problemy z:

- ustalaniem kolejności zdarzeń,
- synchronizacją,
- wykrywaniem awarii,
- spójnością danych.

W systemach równoległych na jednej maszynie synchronizacja jest zwykle łatwiejsza, bo wszystko działa w ramach jednego systemu.

Dlaczego równoległe nie zawsze jest szybsze?

Przetwarzanie równoległe może być szybsze, ale nie zawsze.

Ograniczenia to między innymi:

- narzut tworzenia wątków,
- synchronizacja,
- konflikty dostępu do danych,
- część sekwencyjna programu,
- nierówny podział pracy.

Podobnie przetwarzanie rozproszone może być bardzo wydajne, ale komunikacja przez sieć jest zwykle wolniejsza niż dostęp do pamięci lokalnej.

Najważniejsza intuicja

Przetwarzanie równoległe kojarzymy głównie z wieloma rdzeniami jednej maszyny i pamięcią wspólną, a rozproszone z wieloma komputerami, pamięcią rozproszoną i komunikacją przez sieć.

Pytanie 14: Co wykonuje dyrektywa `#pragma omp single` w kontekście OpenMP?

Poprawna odpowiedź:

a. Uruchamia blok kodu tylko w jednym wątku

Pojęcia do notatki

`#pragma omp single`

Dyrektywa:

```
#pragma omp single
{
    // kod wykonany tylko przez jeden wątek
}
```

oznacza, że dany blok kodu zostanie wykonany **tylko przez jeden wątek** z zespołu wątków.

Pozostałe wątki pomijają ten blok.

Do czego służy `single`?

`single` stosuje się wtedy, gdy pewien fragment programu powinien zostać wykonany tylko raz, mimo że znajduje się wewnątrz regionu równoległego.

Przykład:

```
#pragma omp parallel
{
    #pragma omp single
    {
        printf("Ten komunikat wypisze tylko jeden wątek.\n");
    }

    // dalsza praca równoległa
}
```

`single` a liczba wątków

`#pragma omp single` **nie ogranicza liczby wątków do jednego**.

Wątki nadal istnieją, ale tylko jeden z nich wykonuje oznaczony blok kodu.

Czyli:

parallel → działa wiele wątków

single → tylko jeden z tych wątków wykonuje wskazany blok

single a critical

single i critical są różne.

single oznacza:

jeden wątek wykonuje blok **raz**.

critical oznacza:

wiele wątków może chcieć wykonać blok, ale **tylko jeden naraz**.

Przykład różnicy:

```
#pragma omp single
{
    inicjalizacja();
}
```

Kod wykona się tylko raz.

```
#pragma omp critical
{
    suma += wynik;
}
```

Kod może wykonać wiele wątków, ale po kolei.

Dlaczego pozostałe odpowiedzi są błędne?

b. Ogranicza liczbę wątków do jednego — nie, liczba wątków się nie zmienia.

c. Działa jak #pragma omp critical — nie, critical chroni dostęp wielu wątków do sekcji krytycznej, a single wybiera jeden wątek do wykonania bloku.

d. Tworzy wątek, który nie wykonuje żadnej pracy — nie, single nie tworzy nowego wątku.

Najważniejsza intuicja

#pragma omp single oznacza: z całej grupy wątków tylko jeden ma wykonać ten konkretny fragment kodu.

Pytanie 15: Wypisz zależności pojawiające się w kodzie:

```
1: y = 4 * sin(2 * 3.14);
2: x = 2 * i + z * y;
3: z = x * y;
```

Zależności w kodzie

Nr linii, nr linii Zmienna Typ zależności

1 → 2	y	RAW — Read After Write
1 → 3	y	RAW — Read After Write
2 → 3	x	RAW — Read After Write
2 → 3	z	WAR — Write After Read

Wyjaśnienie

Zależność RAW — Read After Write

RAW oznacza, że jedna instrukcja najpierw zapisuje zmienną, a późniejsza instrukcja ją odczytuje.

Przykład z kodu:

```
1: y = 4 * sin(2 * 3.14);  
2: x = 2 * i + z * y;
```

Linia 1 zapisuje y, a linia 2 odczytuje y.

Czyli:

1 → 2 na zmiennej y: RAW

Podobnie:

```
2: x = 2 * i + z * y;  
3: z = x * y;
```

Linia 2 zapisuje x, a linia 3 odczytuje x.

Czyli:

2 → 3 na zmiennej x: RAW

Zależność WAR — Write After Read

WAR oznacza, że jedna instrukcja najpierw odczytuje zmienną, a późniejsza instrukcja ją zapisuje.

Przykład:

```
2: x = 2 * i + z * y;  
3: z = x * y;
```

Linia 2 odczytuje starą wartość z, a linia 3 zapisuje nową wartość do z.

Czyli:

2 → 3 na zmiennej z: WAR

Najważniejsza intuicja

W tym kodzie najważniejsze są zależności RAW przez y i x, bo późniejsze instrukcje potrzebują wyników wcześniejszych, oraz zależność WAR na z, bo linia 2 musi odczytać stare z, zanim linia 3 je nadpisze.

Pytanie 16: Jakie będą wartości zmiennych a, b i c w dowolnym wątku?

Kod:

```
int a = 3;
int b = 2;
int c = 1;

#pragma omp parallel firstprivate(a) num_threads(4)
{
    int b = a * c;

    #pragma omp barrier

    #pragma omp atomic
    c += 2;
    a += 3;
    b += 4

    #pragma omp barrier
}
```

Odpowiedź dla dowolnego wątku wewnątrz regionu równoległego

a = 6
b = 7
c = 9

Wyjaśnienie

Zmienna a

#pragma omp parallel firstprivate(a)

firstprivate(a) oznacza, że każdy wątek dostaje własną kopię a, ale z wartością początkową z oryginalnej zmiennej.

Na początku w każdym wątku:

a = 3

Potem każdy wątek wykonuje:

a += 3;

więc w każdym wątku:

a = 6

Zmienna b

Na zewnątrz mamy:

int b = 2;

ale wewnątrz regionu równoległego jest nowa lokalna zmienna:

int b = a * c;

Ta zmienna b jest osobna dla każdego wątku.

Na początku:

a = 3

c = 1

więc:

b = 3 * 1 = 3

Potem każdy wątek wykonuje:

b += 4;

czyli:

b = 3 + 4 = 7

Zmienna c

Zmienna c nie jest oznaczona jako private, więc jest **współdzielona** między wszystkimi wątkami.

Na początku:

c = 1

Mamy 4 wątki:

num_threads(4)

Każdy wykonuje atomowo:

#pragma omp atomic

c += 2;

Czyli łącznie:

c = 1 + 4 * 2 = 9

Uwaga

Po wyjściu z regionu równoległego, w kodzie sekwencyjnym wartości zmiennych zewnętrznych byłyby:

$a = 3$

$b = 2$

$c = 9$

bo a było firstprivate, a wewnętrzne b zastępowało zewnętrzne b .

Najważniejsza intuicja

W wątku: a jest prywatną kopią, b jest lokalne dla bloku, a c jest wspólne i zwiększane atomowo przez wszystkie 4 wątki.

Pytanie 17: Zanalizuj zależności przenoszone w pętli. Wpisz obok typu zależności nazwę tablicy której dotyczy, jeśli nie ma wpisz kreske

WAW

WAR

RAW

Kod:

```
for (i = 1; i < N; i++)
{
    A[i] = D[i-1] - C[i-1];
    D[i-1] = 2 * D[i+1];
}
```

Poprawna odpowiedź

zależności wyjścia (WAW): -

antyzależności (WAR): D

zależności rzeczywiste (RAW): -

Analiza instrukcji

Oznaczmy instrukcje:

S1: $A[i] = D[i-1] - C[i-1];$

S2: $D[i-1] = 2 * D[i+1];$

S1

$A[i] = D[i-1] - C[i-1];$

- zapisuje: $A[i]$
- odczytuje: $D[i-1], C[i-1]$

S2

$D[i-1] = 2 * D[i+1];$

- zapisuje: $D[i-1]$
- odczytuje: $D[i+1]$

WAW — Write After Write

WAW oznacza, że dwie iteracje zapisują do tego samego miejsca.

Tutaj:

$A[i]$

jest zapisywane dla różnych wartości i , więc każdy zapis trafia w inny element tablicy.

$D[i-1]$

również jest zapisywane dla różnych wartości i , czyli też do różnych elementów.

Dlatego:

WAW: -

RAW — Read After Write

RAW oznacza, że późniejsza iteracja odczytuje wartość zapisaną przez wcześniejszą iterację.

Tutaj wcześniejsza iteracja zapisuje:

$D[i-1]$

ale późniejsze iteracje nie odczytują tego samego elementu jako wyniku wcześniejszego zapisu.

Dlatego:

RAW: -

WAR — Write After Read

WAR oznacza, że wcześniejsza iteracja odczytuje element, który późniejsza iteracja zapisze.

W instrukcji S2 mamy odczyt:

$D[i+1]$

oraz zapis:

$D[i-1]$

Sprawdźmy przykład:

$i = 1 \rightarrow$ odczyt $D[2]$

$i = 3 \rightarrow$ zapis $D[2]$

Czyli iteracja wcześniejsza odczytuje $D[2]$, a późniejsza iteracja zapisuje do $D[2]$.

To jest zależność:

WAR na tablicy D

Najważniejsza intuicja

W tej pętli występuje zależność WAR na tablicy D, bo wcześniejsza iteracja czyta element $D[i+1]$, który późniejsza iteracja może nadpisać jako $D[i-1]$.

Pytanie 18: Które stwierdzenia dotyczące wątków w Javie są prawdziwe?

Poprawne odpowiedzi:

- a. `Thread.sleep(long millis)` wprowadza wątek w stan oczekiwania na określony czas, ale nie uwalnia blokad, które wątek może posiadać.**
- b. `Thread.yield()` powoduje, że wątek oddaje procesor planisty, umożliwiając wykonanie innych wątków.**
- d. Metoda `join()` w klasie `Thread` blokuje bieżący wątek do momentu, aż wątek, na którym wywołano `join()`, zakończy swoje działanie.**

Niepoprawne odpowiedzi:

- c. `synchronized` na metodzie statycznej synchronizuje dostęp do niej na poziomie instancji klasy — błędne, bo metoda statyczna synchronizuje się na poziomie obiektu klasy, czyli `Class`, a nie konkretnej instancji.**
 - e. Klasa `Runnable` ma metodę `run()`, która musi być jawnie wywołana przez użytkownika — błędne. Metodę `run()` implementujemy, ale zwykle nie wywołujemy jej ręcznie. Uruchamiamy wątek przez `start()`.**
-

Pojęcia do notatki

`Thread.sleep()`

Metoda:

```
Thread.sleep(1000);
```

usypia bieżący wątek na określony czas, tutaj na 1000 ms, czyli 1 sekundę.

Ważne: `sleep()` **nie zwalnia blokad**.

Przykład:

```
synchronized (lock) {  
    Thread.sleep(1000);  
}
```

Wątek śpi, ale nadal trzyma blokadę `lock`.

Thread.yield()

Metoda:

```
Thread.yield();
```

sugeruje planiście systemu, że bieżący wątek może ustąpić procesora innym wątkom.

Nie daje jednak gwarancji, że inny wątek natychmiast się wykona.

To bardziej „podpowiedź” dla planisty niż twarde polecenie.

join()

Metoda:

```
thread.join();
```

powoduje, że bieżący wątek czeka, aż wątek thread zakończy działanie.

Przykład:

```
Thread t = new Thread(() -> {  
    System.out.println("Praca wątku");  
});
```

```
t.start();
```

```
t.join();
```

```
System.out.println("Ten kod wykona się po zakończeniu t");
```

join() jest używane, gdy chcemy zsynchronizować zakończenie pracy wielu wątków.

synchronized na metodzie statycznej

Dla metody statycznej:

```
public static synchronized void metoda() {  
    // kod  
}
```

blokada zakładana jest na obiekcie klasy, czyli na:

```
NazwaKlasy.class
```

Nie jest to blokada konkretnego obiektu utworzonego przez new.

Dla porównania metoda niestatyczna:

```
public synchronized void metoda() {  
    // kod  
}
```

blokuje konkretną instancję, czyli this.

Runnable i metoda run()

Interfejs Runnable wymaga zdefiniowania metody:

```
public void run() {  
    // kod wątku  
}
```

Ale żeby uruchomić ją jako osobny wątek, używamy:

```
Thread t = new Thread(runnable);  
t.start();
```

Nie robimy zwykle:

```
runnable.run();
```

bo wtedy kod wykona się zwyczajnie w bieżącym wątku, a nie w nowym.

Najważniejsza intuicja

W Javie `sleep()` usypia wątek, ale nie zwalnia blokad, `yield()` tylko sugeruje oddanie procesora, a `join()` pozwala poczekać na zakończenie innego wątku.

Pytanie 19: Co umożliwia klauzula `private` w OpenMP?

Poprawna odpowiedź:

a. Każdy wątek otrzymuje własną kopię zmiennej

Pojęcia do notatki

Klauzula `private`

Klauzula `private` w OpenMP sprawia, że wskazana zmienna staje się **prywatna dla każdego wątku**.

Przykład:

```
#pragma omp parallel private(x)  
{  
    x = omp_get_thread_num();  
}
```

Każdy wątek ma własną kopię zmiennej `x`, więc nie nadpisuje wartości zapisanych przez inne wątki.

Zmienna prywatna

Zmienna prywatna istnieje osobno w każdym wątku.

Schemat:

Wątek 0 → własne x
Wątek 1 → własne x
Wątek 2 → własne x
Wątek 3 → własne x

Dzięki temu zmienna może być używana lokalnie przez każdy wątek bez konfliktów dostępu.

Wartość początkowa zmiennej prywatnej

Zmienna oznaczona jako `private` **nie przejmuje automatycznie wartości początkowej** zmiennej oryginalnej.

Jej wartość początkowa jest nieokreślona, dopóki wątek sam jej nie przypisze.

Jeżeli chcemy, aby każdy wątek dostał kopię z wartością początkową, używa się:

`firstprivate(x)`

Dlaczego pozostałe odpowiedzi są błędne?

b. Każdy wątek korzysta z tej samej zmiennej — to opis zmiennej współdzielonej, czyli `shared`.

c. Ustala dostęp tylko dla wątku głównego — do kodu wykonywanego przez wątek główny służy raczej `master` lub `masked`.

d. Zmienia zakres zmiennej na globalny — `private` nie robi zmiennej globalnej.

e. Zabrania dostępu do zmiennej dla innych wątków — nie chodzi o zakaz dostępu, tylko o utworzenie osobnych kopii.

Najważniejsza intuicja

`private(x)` oznacza: każdy wątek ma swoje własne `x`, więc nie współdzieli tej zmiennej z innymi wątkami.

Pytanie 20: W jaki sposób można zminimalizować narzut czasowy związany z tworzeniem wątków?

Poprawna odpowiedź:

a. Poprzez zastosowanie puli wątków

Pojęcia do notatki

Narzut tworzenia wątków

Tworzenie nowego wątku nie jest darmowe. System musi między innymi:

- przydzielić pamięć na stos wątku,
- utworzyć struktury systemowe,
- zarejestrować wątek w planisty systemu,

- rozpocząć jego wykonanie.

Jeśli program często tworzy i niszczy wątki, może tracić dużo czasu na samo zarządzanie nimi.

Pula wątków

Pula wątków, czyli **thread pool**, to zestaw wcześniej utworzonych wątków, które czekają na zadania.

Zamiast tworzyć nowy wątek dla każdego zadania, program przekazuje zadanie do puli, a jeden z istniejących wątków je wykonuje.

Schemat:

Zadania → kolejka zadań → pula gotowych wątków

Dzięki temu zmniejsza się koszt tworzenia i niszczenia wątków.

Dlaczego pula wątków zmniejsza narzut?

Wątki są tworzone raz, a potem wielokrotnie wykorzystywane.

Przykład:

Bez puli:

zadanie 1 → utwórz wątek → wykonaj → usuń wątek

zadanie 2 → utwórz wątek → wykonaj → usuń wątek

Z pulą:

utwórz kilka wątków raz

zadanie 1 → użyj gotowego wątku

zadanie 2 → użyj gotowego wątku

To jest szczególnie korzystne, gdy mamy dużo krótkich zadań.

Dlaczego pozostałe odpowiedzi są błędne?

b. Algorytm Round Robin — dotyczy sposobu planowania czasu procesora, a nie zmniejszania kosztu tworzenia wątków.

c. Ograniczenie liczby wątków do jednego — zmniejszyłoby narzut, ale usuwa równoległość, więc nie jest właściwą metodą optymalizacji programu równoległego.

d. Model aktorów — może korzystać z puli wątków, ale sam w sobie nie jest bezpośrednim mechanizmem minimalizacji kosztu tworzenia wątków.

e. Algorytmy lock-free — zmniejszają narzut synchronizacji i blokowania, ale nie rozwiązują bezpośrednio problemu tworzenia wątków.

Najważniejsza intuicja

Najlepszym sposobem ograniczenia kosztu tworzenia wątków jest utworzenie ich raz i wielokrotne używanie, czyli zastosowanie puli wątków.

TEST 3

Pytanie: Wykonywanie współbieżne oznacza, że:

Poprawne odpowiedzi:

- a. Dwa lub więcej procesów lub wątków wykonuje się / współpracuje w celu rozwiązania jednego zadania
- e. Może być realizowane zarówno na jednym procesorze, jak i na wielu

Niepoprawne odpowiedzi:

- b. Procesy wykonywane są wyłącznie sekwencyjnie — nie, współbieżność oznacza przeplatanie lub równoczesny postęp wielu zadań.
- c. Wszystkie procesy muszą być wykonywane równolegle na wielu rdzeniach — nie muszą. Współbieżność może działać nawet na jednym rdzeniu.
- d. Każdy proces wykonuje się w izolacji, bez możliwości komunikacji — nie, procesy lub wątki współbieżne mogą się komunikować i synchronizować.

Pojęcia do notatki

Wykonywanie współbieżne

Wykonywanie współbieżne oznacza, że kilka procesów lub wątków jest aktywnych w tym samym okresie czasu.

Nie oznacza to koniecznie, że wykonują się dokładnie w tej samej chwili. Na jednym procesorze system może szybko przełączać się między nimi.

Przykład:

czas →

Wątek 1: działa --- czeka --- działa

Wątek 2: czeka --- działa --- czeka

Z perspektywy użytkownika oba wątki „postępują” współbieżnie.

Współbieżność a równoległość

Współbieżność oznacza, że kilka zadań jest obsługiwanych w tym samym czasie logicznym.

Równoległość oznacza, że kilka zadań faktycznie wykonuje się jednocześnie, np. na kilku rdzeniach.

Czyli:

współbieżność → zadania mogą się przeplatać

równoległość → zadania wykonują się jednocześnie fizycznie

Każde wykonanie równoległe jest współbieżne, ale nie każde współbieżne jest równoległe.

Współbieżność na jednym procesorze

Współbieżność może istnieć na jednym procesorze dzięki przełączaniu kontekstu.

Procesor wykonuje przez chwilę jeden wątek, potem drugi, potem kolejny.

Nie ma wtedy prawdziwego równoległego wykonania, ale program nadal jest współbieżny.

Współbieżność na wielu procesorach

Na wielu rdzeniach lub procesorach współbieżne zadania mogą wykonywać się naprawdę jednocześnie.

Wtedy mamy również **równoległość**.

Najważniejsza intuicja

Współbieżność oznacza, że kilka zadań robi postępy w tym samym czasie logicznym; może to być przeplatanie na jednym rdzeniu albo rzeczywiste równoległe wykonanie na wielu rdzeniach.

Pytanie: Do modeli przetwarzania według taksonomii Flynna należą:

Poprawna odpowiedź:

a. SISD, SIMD, MISD, MIMD

Pojęcia do notatki

Taksonomia Flynna

Taksonomia Flynna to klasyfikacja architektur komputerowych według tego, ile strumieni instrukcji i ile strumieni danych jest przetwarzanych jednocześnie.

Opiera się na dwóch pojęciach:

Instruction stream → strumień instrukcji

Data stream → strumień danych

Z połączenia tych dwóch kryteriów powstają cztery modele:

SISD

SIMD

MISD

MIMD

SISD — Single Instruction, Single Data

SISD oznacza: jeden strumień instrukcji i jeden strumień danych.

To klasyczny model przetwarzania sekwencyjnego.

Przykład:

jeden procesor wykonuje jeden program na jednym zbiorze danych

SIMD — Single Instruction, Multiple Data

SIMD oznacza: jeden strumień instrukcji i wiele strumieni danych.

Ta sama instrukcja jest wykonywana równocześnie na wielu danych.

Przykład:

dodaj 1 do wielu elementów tablicy jednocześnie

Typowe zastosowania:

- procesory wektorowe,
 - instrukcje SSE/AVX,
 - GPU.
-

MISD — Multiple Instruction, Single Data

MISD oznacza: wiele strumieni instrukcji i jeden strumień danych.

Jest to rzadko spotykany model.

Może występować w systemach specjalnych, np. odpornych na błędy, gdzie te same dane są przetwarzane różnymi metodami.

MIMD — Multiple Instruction, Multiple Data

MIMD oznacza: wiele strumieni instrukcji i wiele strumieni danych.

Różne procesory lub rdzenie mogą wykonywać różne instrukcje na różnych danych.

Przykłady:

- komputery wielordzeniowe,
 - klastry,
 - systemy rozproszone,
 - programy MPI,
 - programy wielowątkowe.
-

Dlaczego pozostałe odpowiedzi są błędne?

SISO, SIMO, MISO, MIMO to pojęcia znane raczej z systemów komunikacyjnych i antenowych, nie z taksonomii Flynna.

SMR, NUMA, UMA, MPP dotyczą organizacji pamięci lub typów systemów wieloprocessorowych, ale nie są podstawowymi kategoriami Flynna.

RISC, CISC, superskalarność, potokowość dotyczą architektury procesora i sposobu wykonywania instrukcji, ale nie są klasyfikacją Flynna.

Najważniejsza intuicja

Taksonomia Flynna dzieli komputery według liczby strumieni instrukcji i danych: **SISD, SIMD, MISD** oraz **MIMD**.

Pytanie: Wykonanie poza kolejnością, czyli **Out-of-Order Execution**, oznacza:

Poprawne odpowiedzi:

- **b. Procesor może dynamicznie zmieniać kolejność wykonania instrukcji dla optymalizacji wydajności**
 - **e. Może prowadzić do problemów związanych z zależnościami między instrukcjami**
-

Pojęcia do notatki

Out-of-Order Execution

Out-of-Order Execution to technika stosowana w procesorach, która pozwala wykonywać instrukcje w innej kolejności niż ta, w której występują w kodzie programu.

Procesor robi to po to, aby nie czekać bezczynnie, gdy jakaś instrukcja czeka na dane, np. z pamięci.

Przykład:

Instrukcja 1: czeka na dane z pamięci

Instrukcja 2: zależy od instrukcji 1

Instrukcja 3: jest niezależna

Procesor może wykonać instrukcję 3 wcześniej, zamiast czekać na zakończenie instrukcji 1.

Cel wykonania poza kolejnością

Celem jest **zwiększenie wydajności procesora**.

Procesor może lepiej wykorzystać swoje jednostki wykonawcze, wykonując te instrukcje, które są już gotowe.

Dzięki temu można ograniczyć czas bezczynności rdzenia.

Zależności między instrukcjami

Procesor nie może dowolnie przestawiać instrukcji.

Musi uważać na zależności, np.:

$x = a + b;$

$y = x * 2;$

Druga instrukcja zależy od wyniku pierwszej, więc nie może zostać wykonana wcześniej.

Typowe zależności:

- **RAW** — Read After Write,
- **WAR** — Write After Read,
- **WAW** — Write After Write.

Dlaczego pozostałe odpowiedzi są błędne?

a. Kolejność wykonywania instrukcji zawsze musi być zgodna z kolejnością w kodzie — to opis wykonania **in-order**, a nie **out-of-order**.

c. Instrukcje są wykonywane zgodnie z priorytetem nadanym przez programistę — procesor sam decyduje o kolejności na podstawie dostępności danych i zasobów.

d. Każda instrukcja musi być wykonywana na oddzielnym rdzeniu — nie, **out-of-order** dotyczy głównie wykonywania instrukcji wewnątrz jednego rdzenia/procesora.

Najważniejsza intuicja

Out-of-order execution oznacza, że procesor może przestawiać kolejność instrukcji, aby działać szybciej, ale musi zachować poprawność wynikającą z zależności między instrukcjami.

Pytanie: Race condition, czyli **wyścig**, występuje, gdy:

Poprawne odpowiedzi:

- **a. Wynik programu zależy od niezaplanowanej kolejności wykonywania wątków**
- **b. Dwa lub więcej wątków jednocześnie modyfikuje tę samą zmienną bez synchronizacji**
- **e. Może prowadzić do niespójnych lub nieprzewidywalnych wyników**

Niepoprawne odpowiedzi:

- **c. Każdy wątek działa na oddzielnej przestrzeni adresowej** — wtedy ryzyko wyścigu na wspólnej zmiennej jest mniejsze, bo wątki/procesy nie współdzielą bezpośrednio danych.
- **d. Program nie zawiera sekcji krytycznych** — samo to nie oznacza jeszcze wyścigu. Wyścig pojawia się wtedy, gdy jest współdzielony zasób i brak synchronizacji dostępu.

Pojęcia do notatki

Race condition / wyścig

Race condition występuje wtedy, gdy wynik programu zależy od przypadkowej kolejności wykonania operacji przez wątki lub procesy.

Najczęściej chodzi o sytuację, w której kilka wątków jednocześnie odczytuje i zapisuje tę samą zmienną bez odpowiedniej synchronizacji.

Przykład wyścigu

```
licznik++;
```

Ta operacja wygląda jak jedna instrukcja, ale w rzeczywistości może składać się z kilku kroków:

1. odczytaj wartość licznik
2. dodaj 1
3. zapisz nową wartość

Jeśli dwa wątki wykonają to jednocześnie, oba mogą odczytać tę samą starą wartość i zapisać ten sam wynik. Wtedy jedno zwiększenie zostanie utracone.

Sekcja krytyczna

Sekcja krytyczna to fragment kodu, w którym wątek korzysta ze współdzielonego zasobu, np. zmiennej, tablicy, pliku albo bufora.

Aby uniknąć race condition, sekcję krytyczną zabezpiecza się np. przez:

```
#pragma omp critical
```

albo w Pthreads:

```
pthread_mutex_lock(&mutex);  
// sekcja krytyczna  
pthread_mutex_unlock(&mutex);
```

Najważniejsza intuicja

Race condition pojawia się wtedy, gdy kilka wątków ściga się o dostęp do tych samych danych, a wynik zależy od tego, który wątek wykona operację pierwszy.

Pytanie: Jakie są możliwe rozwiązania problemu uczących filozofów?

Poprawne odpowiedzi:

- a. Ograniczenie liczby filozofów mogących jednocześnie jeść
- b. Nadanie priorytetów filozofom, aby uniknąć zagłodzenia
- d. Wprowadzenie globalnego semafora ograniczającego liczbę jednocześnie jedzących filozofów
- e. Użycie strategii, w której filozof podnosi oba widelce jednocześnie lub żaden

Niepoprawna odpowiedź:

- **c. Pozostawienie filozofów bez żadnej synchronizacji** — prowadziłoby do zakleszczeń, wyścigów albo zagłodzenia.
-

Pojęcia do notatki

Problem uczujących filozofów

Problem uczujących filozofów to klasyczny problem synchronizacji w programowaniu współbieżnym.

Kilku filozofów siedzi przy stole. Każdy potrzebuje dwóch widelców, aby jeść: lewego i prawego. Widelec jest współdzielony z sąsiadem.

Problem pokazuje, co może się stać, gdy wiele wątków konkuruje o ograniczone zasoby.

Zakleszczenie — deadlock

Deadlock występuje wtedy, gdy każdy filozof podniesie jeden widelec i czeka na drugi.

Przykład:

Filozof 1 trzyma lewy widelec i czeka na prawy

Filozof 2 trzyma lewy widelec i czeka na prawy

Filozof 3 trzyma lewy widelec i czeka na prawy

...

Nikt nie może kontynuować, bo każdy czeka na zasób trzymany przez kogoś innego.

Zagłodzenie — starvation

Starvation oznacza, że dany wątek bardzo długo albo nigdy nie dostaje dostępu do zasobu.

W problemie filozofów oznacza to sytuację, w której jeden filozof ciągle przegrywa dostęp do widelców i nie może jeść.

Semafor globalny

Jednym z rozwiązań jest użycie semafora ograniczającego liczbę filozofów, którzy mogą próbować jeść jednocześnie.

Na przykład dla N filozofów można dopuścić maksymalnie $N - 1$ filozofów do próby podnoszenia widelców.

Dzięki temu nie powstanie sytuacja, w której wszyscy jednocześnie trzymają po jednym widelcu i czekają na drugi.

Podnoszenie obu widelców naraz

Inna strategia polega na tym, że filozof może podnieść **oba widelce jednocześnie**, albo nie podnosi żadnego.

Czyli nie może dojść do sytuacji:
mam jeden widelec i czekam na drugi
To eliminuje jedną z przyczyn zakleszczenia.

Priorytety

Można też stosować priorytety lub mechanizmy sprawiedliwego dostępu.
Celem jest uniknięcie sytuacji, w której jeden filozof jest stale pomijany.
Przykład: filozof, który długo czeka, dostaje wyższy priorytet.

Najważniejsza intuicja

Problem uczujących filozofów pokazuje, że samo współdzielenie zasobów nie wystarczy — trzeba dodać synchronizację, aby uniknąć zakleszczenia i zagłodzenia.

Pytanie: W języku C, w bibliotekach POSIX, mutex służy do:

Poprawne odpowiedzi:

- **a. Zapewnienia wzajemnego wykluczania przy dostępie do zasobów współdzielonych**
- **b. Ochrony sekcji krytycznych przed jednoczesnym dostępem wielu wątków**
- **d. Synchronizacji operacji poprzez blokowanie wątku do momentu zwolnienia zasobu**

Niepoprawne odpowiedzi:

- **c. Wymiany informacji między wątkami bez konieczności blokowania** — do komunikacji między wątkami służą raczej zmienne warunkowe, kolejki, semafony itp.
 - **e. Przyspieszania wykonywania kodu przez równoległe wykonywanie instrukcji** — mutex nie przyspiesza programu; często wręcz wprowadza narzut, ale zapewnia poprawność.
-

Pojęcia do notatki

Mutex

Mutex, czyli **mutual exclusion**, to mechanizm wzajemnego wykluczania.

Służy do tego, aby tylko jeden wątek naraz mógł korzystać z chronionego zasobu.

Przykład zasobu współdzielonego:

```
int licznik;
```

Jeśli wiele wątków jednocześnie wykonuje:

```
licznik++;
```

może dojść do błędnego wyniku. Mutex zabezpiecza taki fragment kodu.

Sekcja krytyczna

Sekcja krytyczna to fragment kodu, w którym wątek korzysta ze wspólnych danych.

Przykład w Pthreads:

```
pthread_mutex_lock(&mutex);
```

```
// sekcja krytyczna  
licznik++;
```

```
pthread_mutex_unlock(&mutex);
```

Tylko jeden wątek może znajdować się między `pthread_mutex_lock` a `pthread_mutex_unlock`.

Blokowanie wątku

Jeżeli jeden wątek już zablokował mutex, to drugi wątek próbujący wykonać:

```
pthread_mutex_lock(&mutex);
```

musi poczekać.

Wątek zostaje zablokowany do momentu, aż mutex zostanie zwolniony przez:

```
pthread_mutex_unlock(&mutex);
```

Najważniejsza intuicja

Mutex działa jak klucz do sekcji krytycznej: tylko jeden wątek może mieć klucz naraz, a reszta musi czekać.

Pytanie: Zmienne warunkowe w POSIX, czyli **condition variables**, służą do:

Poprawne odpowiedzi:

- **a. Synchronizacji między wątkami poprzez umożliwienie wątkowi czekania na określony stan**
- **c. Unikania aktywnego czekania, czyli busy-wait, w pętlach synchronizacyjnych**
- **d. Budzenia jednego lub wielu oczekujących wątków w odpowiednim momencie**

Niepoprawne odpowiedzi:

- **b. Blokowania dostępu do pamięci współdzielonej na stałe** — do ochrony dostępu służy mutex, a nie zmienna warunkowa.
 - **e. Definiowania priorytetów wątków w systemie POSIX** — zmienne warunkowe nie służą do ustawiania priorytetów.
-

Pojęcia do notatki

Zmienna warunkowa

Zmienna warunkowa pozwala wątkowi czekać, aż zostanie spełniony określony warunek.

Przykład sytuacji:

Wątek konsument czeka, aż w buforze pojawią się dane.

Wątek producent dodaje dane i budzi konsumenta.

pthread_cond_wait()

Funkcja:

```
pthread_cond_wait(&cond, &mutex);
```

powoduje, że wątek zasypia i czeka na sygnał.

Ważne: podczas czekania pthread_cond_wait() zwalnia mutex, aby inne wątki mogły zmienić stan programu.

Typowy schemat:

```
pthread_mutex_lock(&mutex);
```

```
while (!warunek) {  
    pthread_cond_wait(&cond, &mutex);  
}
```

```
// kod wykonywany po spełnieniu warunku
```

```
pthread_mutex_unlock(&mutex);
```

Używa się while, ponieważ po przebudzeniu trzeba ponownie sprawdzić warunek.

Unikanie busy-wait

Bez zmiennych warunkowych wątek mógłby stale sprawdzać warunek:

```
while (!warunek) {  
    // aktywne czekanie  
}
```

To marnuje czas procesora.

Zmienna warunkowa pozwala wątkowi spać, aż inny wątek go obudzi.

pthread_cond_signal() i pthread_cond_broadcast()

Do budzenia wątków służą:

```
pthread_cond_signal(&cond);
```

Budzi **jeden** oczekujący wątek.

```
pthread_cond_broadcast(&cond);
```

Budzi **wszystkie** oczekujące wątki.

Najważniejsza intuicja

Zmienna warunkowa pozwala wątkowi spać do momentu spełnienia warunku, zamiast marnować procesor na ciągłe sprawdzanie.

Pytanie 8: W MPI komunikacja między procesami odbywa się za pomocą:

Poprawne odpowiedzi:

- a. Modelu „send-receive”
- b. Komunikatorów, takich jak `MPI_COMM_WORLD`
- d. Grupowych operacji przesyłania, takich jak `MPI_Bcast` i `MPI_Gather`

Niepoprawne odpowiedzi:

- c. Wspólnej pamięci, bez konieczności przesyłania komunikatów — MPI opiera się głównie na przesyłaniu komunikatów, a nie na pamięci wspólnej.
 - e. Mechanizmów dostępu do plików w systemie lokalnym — dostęp do plików nie jest podstawowym mechanizmem komunikacji między procesami MPI.
-

Pojęcia do notatki

MPI

MPI, czyli **Message Passing Interface**, to standard komunikacji między procesami w programach równoległych i rozproszonych.

MPI jest często używane w systemach z pamięcią rozproszoną, gdzie każdy proces ma własną pamięć i musi jawnie wysłać dane do innych procesów.

Model send-receive

Podstawowy sposób komunikacji w MPI to wysyłanie i odbieranie komunikatów.

Przykład:

```
MPI_Send(...);
```

```
MPI_Recv(...);
```

`MPI_Send` wysyła dane z jednego procesu, a `MPI_Recv` odbiera dane w innym procesie.

Schemat:

Proces 0 -- MPI_Send --> Proces 1
Proces 1 -- MPI_Recv <-- Proces 0

Komunikator

Komunikator określa grupę procesów, które mogą się ze sobą komunikować.

Najbardziej znany komunikator to:

MPI_COMM_WORLD

Obejmuje on wszystkie procesy uruchomione w danym programie MPI.

Przykład:

```
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Operacje grupowe

MPI udostępnia także komunikację zbiorową, czyli operacje wykonywane przez wiele procesów naraz.

Przykłady:

```
MPI_Bcast(...);  
MPI_Gather(...);
```

MPI_Bcast rozsyła dane z jednego procesu do wszystkich.

jeden → wielu

MPI_Gather zbiera dane z wielu procesów do jednego.

wielu → jeden

Najważniejsza intuicja

MPI komunikuje procesy przez jawne przesyłanie komunikatów: pojedynczo przez MPI_Send/MPI_Recv albo zbiorowo przez funkcje takie jak MPI_Bcast i MPI_Gather.

Pytanie 9: Procedury przesyłania komunikatów w MPI można podzielić na:

Poprawne odpowiedzi:

- a. Blokujące, np. MPI_Send, MPI_Recv
- b. Nieblokujące, np. MPI_Isend, MPI_Irecv
- d. Punkt-do-punktu, np. MPI_Send, MPI_Recv, i grupowe, np. MPI_Bcast, MPI_Gather
- e. Operacje z użyciem MPI_Request, pozwalające na asynchroniczne przesyłanie

Niepoprawna odpowiedź:

- **c. Wyłącznie na procedury synchroniczne, ponieważ nie istnieją metody asynchroniczne** — błędne, bo MPI posiada komunikację nieblokującą/asynchroniczną, np. MPI_Isend i MPI_Irecv.

Pojęcia do notatki

Komunikacja blokująca w MPI

Komunikacja **blokująca** oznacza, że funkcja nie kończy się, dopóki operacja komunikacyjna nie osiągnie stanu pozwalającego bezpiecznie kontynuować program.

Przykład:

```
MPI_Send(...);  
MPI_Recv(...);
```

MPI_Recv zwykle czeka, aż wiadomość zostanie odebrana.

Komunikacja nieblokująca w MPI

Komunikacja **nieblokująca** rozpoczyna wysyłanie lub odbieranie danych, ale program może wykonywać dalsze instrukcje bez czekania na zakończenie komunikacji.

Przykład:

```
MPI_Isend(..., &request);  
MPI_Irecv(..., &request);
```

Do sprawdzenia lub oczekiwania na zakończenie używa się później np.:

```
MPI_Wait(&request, &status);
```

MPI_Request

MPI_Request to uchwyt opisujący rozpoczętą operację nieblokującą.

Pozwala MPI śledzić, czy dana operacja, np. MPI_Isend albo MPI_Irecv, została już zakończona.

Typowy schemat:

```
MPI_Request request;
```

```
MPI_Isend(..., &request);
```

```
// tutaj można wykonywać inne obliczenia
```

```
MPI_Wait(&request, MPI_STATUS_IGNORE);
```

Komunikacja punkt-do-punktu

Komunikacja **punkt-do-punktu** odbywa się między dwoma konkretnymi procesami.

Przykład:

Proces 0 → Proces 1

Typowe funkcje:

MPI_Send(...);

MPI_Recv(...);

Komunikacja grupowa

Komunikacja **grupowa**, czyli zbiorowa, obejmuje wiele procesów naraz.

Przykłady:

MPI_Bcast(...);

MPI_Gather(...);

MPI_Bcast rozsyła dane z jednego procesu do wszystkich.

MPI_Gather zbiera dane od wielu procesów do jednego.

Najważniejsza intuicja

W MPI komunikację można klasyfikować jako blokującą lub nieblokującą oraz jako punkt-do-punktu lub zbiorową; operacje nieblokujące korzystają z MPI_Request, aby później sprawdzić ich zakończenie.

Pytanie 10: Jakie operacje można wykonać na plikach w MPI-IO?

Poprawne odpowiedzi:

- a. MPI_File_open()
- b. MPI_File_read()
- d. MPI_File_write_at()
- e. MPI_File_close()

Niepoprawna odpowiedź:

- c. MPI_File_lock() — taka funkcja nie jest standardową operacją MPI-IO. W MPI istnieją funkcje blokujące np. dla RMA, takie jak MPI_Win_lock(), ale nie MPI_File_lock().

Pojęcia do notatki

MPI-IO

MPI-IO to część standardu MPI służąca do równoległych operacji wejścia-wyjścia na plikach.

Pozwala wielu procesom MPI wspólnie otwierać, czytać i zapisywać dane do plików.

MPI_File_open()

Funkcja:

MPI_File_open(...)

służy do otwarcia pliku przez procesy MPI.

Można określić tryb otwarcia, np.:

MPI_MODE_RDONLY

MPI_MODE_WRONLY

MPI_MODE_RDWR

MPI_MODE_CREATE

MPI_File_read()

Funkcja:

MPI_File_read(...)

służy do odczytu danych z pliku.

Proces odczytuje dane z aktualnej pozycji w pliku lub zgodnie z ustawionym widokiem pliku.

MPI_File_write_at()

Funkcja:

MPI_File_write_at(...)

służy do zapisu danych w określonym miejscu pliku, podanym jako przesunięcie, czyli **offset**.

Przykład intuicyjny:

Proces 0 zapisuje od pozycji 0

Proces 1 zapisuje od pozycji 100

Proces 2 zapisuje od pozycji 200

Dzięki temu wiele procesów może zapisywać różne fragmenty jednego pliku.

MPI_File_close()

Funkcja:

MPI_File_close(...)

zamyka plik otwarty wcześniej przez MPI-IO.

Po zakończeniu pracy z plikiem należy go zamknąć, żeby zwolnić zasoby i poprawnie zakończyć operacje wejścia-wyjścia.

Najważniejsza intuicja

MPI-IO umożliwia procesom MPI wspólne operacje na plikach: otwieranie, czytanie, zapisywanie w określonym miejscu i zamykanie plików.

Pytanie 11: Interkomunikatory w MPI służą do:

Poprawne odpowiedzi:

- **a. Komunikacji między niezależnymi grupami procesów MPI**
- **b. Obsługi dynamicznego tworzenia nowych procesów, np. `MPI_Comm_spawn`**
- **d. Zarządzania / organizowania hierarchii procesów, np. w architekturze master-worker**
Z doprecyzowaniem: interkomunikator sam nie „zarządza” procesami automatycznie, ale umożliwia komunikację między grupami, co można wykorzystać w układzie master-worker.

Niepoprawne odpowiedzi:

- **c. Przesyłania danych tylko w ramach jednego procesu** — MPI służy do komunikacji między procesami.
- **e. Synchronizacji wątków w aplikacji wielowątkowej** — to dotyczy np. mutexów, semaforów, OpenMP lub Pthreads, a nie interkomunikatorów MPI.

Pojęcia do notatki

Interkomunikator

Interkomunikator w MPI służy do komunikacji między **dwiema oddzielnymi grupami procesów**.

Schemat:

Grupa A: P0, P1, P2



Grupa B: P3, P4, P5

Interkomunikator łączy te dwie grupy i pozwala im wymieniać komunikaty.

Różnica: intrakomunikator a interkomunikator

Intrakomunikator służy do komunikacji wewnątrz jednej grupy procesów.

Przykład:

`MPI_COMM_WORLD`

obejmuje wszystkie procesy programu MPI.

Interkomunikator służy natomiast do komunikacji między dwiema różnymi grupami procesów.

`MPI_Comm_spawn`

Funkcja:

`MPI_Comm_spawn(...)`

pozwała dynamicznie utworzyć nowe procesy MPI w trakcie działania programu.

Po utworzeniu procesów potomnych MPI tworzy **interkomunikator** między grupą procesów rodzicielskich a grupą procesów potomnych.

Schemat:

Procesy rodzica $\leftrightarrow$ procesy potomne

Master-worker

W architekturze **master-worker** jeden proces lub grupa procesów pełni rolę nadrzędną, a inne procesy wykonują przydzielone zadania.

Interkomunikatory mogą być użyte do oddzielenia i połączenia takich grup, np.:

Master / grupa masterów $\leftrightarrow$ workerzy

Najważniejsza intuicja

Interkomunikator w MPI łączy dwie osobne grupy procesów, np. procesy rodzicielskie i procesy potomne utworzone przez `MPI_Comm_spawn`.

Pytanie 12: Które stwierdzenia dotyczące OpenMP są prawdziwe?

Poprawne odpowiedzi:

- a. OpenMP działa tylko / głównie w systemach z pamięcią wspólną
- b. OpenMP umożliwia zarówno grubo-, jak i drobnoziarnistą równoległość
- d. OpenMP obsługuje model SPMD poprzez tworzenie zespołów wątków

Niepoprawne odpowiedzi:

- c. OpenMP wymaga, aby wszystkie pętle były zrównoleglone — nie, programista wybiera, które pętle lub fragmenty kodu mają być równoległe.
 - e. OpenMP wymaga ręcznego przydzielania pamięci współdzielonej — nie, OpenMP działa w modelu pamięci wspólnej, a zmienne mogą być automatycznie współdzielone lub prywatne zależnie od klauzul.
-

Pojęcia do notatki

OpenMP

OpenMP to standard programowania równoległego dla systemów z **pamięcią wspólną**.

Oznacza to, że wiele wątków działa w ramach jednego procesu i może korzystać ze wspólnej przestrzeni adresowej.

Przykład:

```
#pragma omp parallel
{
    printf("Praca równoległa\n");
}
```

Pamięć wspólna w OpenMP

W OpenMP wątki mogą współdzielić zmienne.

Przykład:

```
int suma = 0;

#pragma omp parallel
{
    // wszystkie wątki widzą zmienną suma
}
```

Nie trzeba ręcznie przesyłać danych między wątkami tak jak w MPI.

Można jednak określać, które zmienne mają być wspólne, a które prywatne:

```
#pragma omp parallel shared(suma) private(i)
```

Równoległość gruboziarnista

Równoległość gruboziarnista oznacza podział programu na większe fragmenty pracy.

Przykład:

```
#pragma omp parallel sections
{
    #pragma omp section
    zadanie_A();

    #pragma omp section
    zadanie_B();
}
```

Każdy wątek może wykonywać większe, osobne zadanie.

Równoległość drobnoziarnista

Równoległość drobnoziarnista oznacza podział pracy na małe części, na przykład iteracje pętli.

Przykład:

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
```

```
    oblicz(i);  
}
```

Tutaj iteracje pętli są dzielone między wątki.

Model SPMD

SPMD, czyli **Single Program, Multiple Data**, oznacza, że wiele wątków wykonuje ten sam program, ale może pracować na różnych danych.

W OpenMP wygląda to tak:

```
#pragma omp parallel  
{  
    int id = omp_get_thread_num();  
    // każdy wątek wykonuje ten sam kod,  
    // ale może pracować na innych danych  
}
```

Każdy wątek wykonuje ten sam fragment kodu, ale może używać własnego numeru wątku i przetwarzać inną część danych.

Najważniejsza intuicja

OpenMP służy do programowania równoległego w pamięci wspólnej: pozwala łatwo dzielić pracę między wątki, zarówno na poziomie dużych zadań, jak i pojedynczych iteracji pętli.

Pytanie 13: Które dyrektywy OpenMP służą do synchronizacji wątków?

Z obrazu widać odpowiedzi:

- a. #pragma omp barrier
- b. #pragma omp critical
- c. #pragma omp atomic
- d. #pragma omp distribute
- e. #pragma omp master

Poprawne odpowiedzi:

- a. #pragma omp barrier
- b. #pragma omp critical
- c. #pragma omp atomic

Niepoprawne / mniej właściwe jako dyrektywy synchronizacji:

- d. #pragma omp distribute — służy do rozdzielania iteracji między zespoły wątków, a nie bezpośrednio do synchronizacji.
- e. #pragma omp master — wskazuje, że blok wykona tylko wątek główny, ale sama dyrektywa nie synchronizuje wszystkich wątków.

Pojęcia do notatki

Synchronizacja w OpenMP

Synchronizacja oznacza kontrolowanie kolejności wykonywania operacji przez wątki, aby uniknąć błędów współbieżności.

Jest potrzebna, gdy wiele wątków:

- korzysta z tych samych danych,
- musi poczekać na siebie nawzajem,
- wykonuje operacje na wspólnej zmiennej.

#pragma omp barrier

Dyrektywa:

```
#pragma omp barrier
```

powoduje, że wszystkie wątki muszą dojść do tego miejsca.

Wątek, który dotrze wcześniej, czeka na pozostałe.

Wątek 0 → czeka

Wątek 1 → czeka

Wątek 2 → czeka

Wątek 3 → wszyscy idą dalej

#pragma omp critical

Dyrektywa:

```
#pragma omp critical
{
    // sekcja krytyczna
}
```

pozwala tylko jednemu wątkowi naraz wykonać dany fragment kodu.

Stosuje się ją do ochrony sekcji krytycznych, np.:

```
#pragma omp critical
{
    suma += lokalny_wynik;
}
```

#pragma omp atomic

Dyrektywa:


```
#pragma omp atomic
x++;
```

zapewnia atomowe wykonanie pojedynczej operacji na zmiennej współdzielonej.

Jest lżejsza niż critical, ale działa tylko dla prostych operacji, takich jak:

```
x++;
x += 2;
suma += wynik;
```

#pragma omp master

Dyrektywa:

```
#pragma omp master
{
    // kod wykonany tylko przez wątek główny
}
```

spawia, że blok kodu wykona tylko **wątek master**, czyli zwykle wątek o numerze 0.

Nie jest to jednak typowa dyrektywa synchronizacyjna, bo pozostałe wątki nie muszą na nią czekać.

Najważniejsza intuicja

Do synchronizacji w OpenMP służą głównie barrier, critical i atomic: barrier każe wątkom czekać na siebie, critical wpuszcza jeden wątek do sekcji krytycznej, a atomic chroni pojedynczą operację na zmiennej.

Pytanie 14: Wartości zmiennych po zakończeniu pętli OpenMP.

Kod z obrazka:

```
int a = 1, b = 4, c = 5, d = 0, i;
```

```
#pragma omp parallel for private(b) shared(c) lastprivate(d) num_threads(3)
for (i = 0; i < 3; i++) {
    b += 1; 4
    c += b; 7 9
    d = omp_get_thread_num(); 2
}
```

Odpowiedź

```
a = 1
b = 4
c = ?
d = 2
```

Z doprecyzowaniem: d = 2 przy typowym statycznym podziale 3 iteracji na 3 wątki. Formalnie wartość d zależy od tego, który wątek wykona ostatnią logiczną iterację i = 2.

Wyjaśnienie

Zmienna a

a nie jest modyfikowana w pętli.

```
int a = 1;
```

Dlatego po pętli:

```
a = 1
```

Zmienna b

b jest oznaczona jako:

```
private(b)
```

To znaczy, że każdy wątek ma własną kopię b.

Zewnętrzna zmienna:

```
b = 4
```

nie jest zmieniana przez pętlę.

Dlatego po pętli:

```
b = 4
```

Uwaga: prywatna kopia b w wątku nie jest automatycznie inicjalizowana wartością 4, bo użyto `private`, a nie `firstprivate`.

Zmienna c

c jest oznaczona jako:

```
shared(c)
```

Czyli wszystkie wątki modyfikują tę samą zmienną:

```
c += b;
```

Nie ma tutaj `atomic`, `critical` ani redukcji, więc występuje **race condition**. Dodatkowo b jako `private` ma nieokreśloną wartość początkową.

Dlatego:

```
c = ?
```

Zmienna d

d jest oznaczona jako:

lastprivate(d)

To znaczy, że po zakończeniu pętli zmienna zewnętrzna d dostaje wartość z **ostatniej logicznej iteracji pętli**, czyli z iteracji:

i = 2

W tej iteracji wykonywane jest:

d = omp_get_thread_num();

Przy 3 iteracjach i 3 wątkach typowo:

i = 0 → wątek 0

i = 1 → wątek 1

i = 2 → wątek 2

więc:

d = 2

Najważniejsza intuicja

private(b) zostawia zewnętrzne b bez zmian, **shared(c)** bez synchronizacji daje wynik losowy, a **lastprivate(d)** przepisuje wartość z ostatniej iteracji pętli.

Pytanie 15: Które stwierdzenia dotyczące modeli równoległości są prawdziwe?

Poprawne odpowiedzi:

- **a. ILP, czyli Instruction Level Parallelism, jest zarządzane na poziomie procesora, a nie bezpośrednio przez programistę**
- **b. Loop Level Parallelism wymaga od programisty lub kompilatora podziału iteracji pętli między procesory**
- **c. Task Level Parallelism polega na przypisaniu różnych części programu do różnych wątków lub procesów**

Niepoprawne odpowiedzi:

- **d. Job Level Parallelism oznacza równoczesne wykonywanie kodu w ramach jednego procesu** — to bardziej pasuje do równoległości wątków lub zadań. Job-level dotyczy raczej równoczesnego wykonywania wielu niezależnych zadań/programów.
- **e. ILP jest kontrolowane wyłącznie przez kod źródłowy programu** — nie, ILP zależy głównie od procesora, kompilatora i zależności między instrukcjami.

Pojęcia do notatki

ILP — Instruction Level Parallelism

ILP, czyli równoległość na poziomie instrukcji, polega na tym, że procesor wykonuje kilka instrukcji jednocześnie lub częściowo nakłada ich wykonanie.

Przykłady mechanizmów ILP:

potokowość
superskalarność
wykonanie poza kolejnością
predykcja skoków

Programista zwykle nie zarządza tym bezpośrednio. Robi to procesor oraz kompilator.

Loop Level Parallelism

Loop Level Parallelism oznacza równoległość na poziomie pętli.

Polega na podziale iteracji pętli między wiele wątków lub procesów.

Przykład w OpenMP:

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    oblicz(i);
}
```

Tutaj iteracje pętli są rozdzielane między dostępne wątki.

Task Level Parallelism

Task Level Parallelism oznacza równoległość na poziomie zadań.

Program dzieli się na różne części, które mogą być wykonywane równocześnie.

Przykład:

Wątek 1 → wczytywanie danych
Wątek 2 → przetwarzanie danych
Wątek 3 → zapis wyników

Każdy wątek lub proces wykonuje inne zadanie.

Job Level Parallelism

Job Level Parallelism oznacza równoległe wykonywanie wielu niezależnych zadań lub programów.

Przykład:

Zadanie 1 → symulacja A
Zadanie 2 → symulacja B
Zadanie 3 → analiza danych C

To często występuje w systemach kolejkowych HPC, np. SLURM, gdzie wiele niezależnych zadań działa na klastrze.

Najważniejsza intuicja

ILP dzieje się głównie wewnątrz procesora, loop-level dzieli iteracje pętli, task-level dzieli program na zadania, a job-level uruchamia wiele niezależnych zadań lub programów.

Pytanie 16: Równoważenie obciążenia w programach równoległych wymaga:

Poprawne odpowiedzi:

- a. Podziału pracy między wątki w taki sposób, aby uniknąć wąskich gardeł
- b. Zapewnienia, że najwolniejszy wątek nie determinuje całkowitego czasu wykonania
- d. Minimalizacji komunikacji między wątkami/procesami

Niepoprawne odpowiedzi:

- c. Wyłączenie statycznego przydzielania zadań, aby uniknąć kosztów dynamicznego zarządzania — nie zawsze. Czasem lepsze jest dynamiczne przydzielanie zadań.
- e. Ignorowania opóźnień wynikających z operacji synchronizacji — nie, opóźnienia synchronizacji trzeba brać pod uwagę.

Pojęcia do notatki

Równoważenie obciążenia

Równoważenie obciążenia, czyli **load balancing**, polega na takim podziale pracy między wątki, procesy lub węzły, aby każdy z nich miał podobną ilość pracy.

Celem jest uniknięcie sytuacji, w której część wątków już skończyła, a jeden nadal pracuje.

Przykład złego podziału:

Wątek 1: 5 sekund pracy

Wątek 2: 5 sekund pracy

Wątek 3: 5 sekund pracy

Wątek 4: 30 sekund pracy

Cały program zakończy się dopiero po 30 sekundach, bo najwolniejszy wątek blokuje zakończenie obliczeń.

Wąskie gardło

Wąskie gardło to fragment programu lub zasób, który ogranicza wydajność całego systemu.

Może nim być:

- przeciążony wątek,
- zbyt duża komunikacja,
- sekcja krytyczna,
- wolny dostęp do pamięci,

- nierówny podział danych.

W równoważeniu obciążenia staramy się tak rozdzielić pracę, żeby żaden wątek nie stał się wąskim gardłem.

Najwolniejszy wątek

W programie równoległym czas wykonania często zależy od tego, kiedy skończy **najwolniejszy wątek**.

Dlatego jeśli jeden wątek dostanie dużo więcej pracy niż pozostałe, cały program będzie czekał właśnie na niego.

Równoważenie obciążenia ma zmniejszyć ten problem.

Statyczny przydział pracy

Stacyjny przydział pracy oznacza, że zadania są rozdzielane przed rozpoczęciem obliczeń.

Przykład:

100 iteracji, 4 wątki → każdy dostaje po 25 iteracji

To działa dobrze, gdy każda iteracja ma podobny koszt.

Jeśli jednak jedne iteracje są dużo cięższe od innych, statyczny podział może prowadzić do nierównowagi.

Dynamiczny przydział pracy

Dynamiczny przydział pracy oznacza, że wątki pobierają kolejne zadania w trakcie działania programu.

Wątek, który skończy szybciej, dostaje następną porcję pracy.

To pomaga przy nierównych zadaniach, ale wprowadza dodatkowy narzut zarządzania.

Minimalizacja komunikacji

Komunikacja między wątkami lub procesami może spowalniać program.

W modelu rozproszonym, np. MPI, przesyłanie danych między procesami jest dużo droższe niż lokalne obliczenia.

Dlatego dobry podział pracy powinien:

- równoważyć obciążenie,
- ograniczać przesyłanie danych,
- zmniejszać liczbę synchronizacji,
- unikać niepotrzebnego czekania.

Najważniejsza intuicja

Równoważenie obciążenia polega na tym, żeby wszystkie wątki miały podobną ilość pracy i żeby żaden z nich nie spowalniał całego programu.

Pytanie 17: Które mechanizmy zapewniają współbieżność w Javie?

Poprawne odpowiedzi:

- a. Klasa Thread oraz interfejs Runnable
- b. Operacje atomowe, takie jak `compareAndSet()` i `getAndIncrement()`
- d. Mechanizmy synchronizacji: zamki Lock, zmienne warunku Condition
- e. Klasy przystosowane do współbieżności, np. `BlockingQueue` i `ConcurrentMap`

Niepoprawna / problematyczna odpowiedź:

- c. Metody `synchronized()` w konstruktorach klas — zapis jest niepoprawny. W Javie istnieje słowo kluczowe `synchronized`, ale nie metoda `synchronized()`. Dodatkowo konstruktorów nie deklaruje się jako `synchronized`.

Pojęcia do notatki

Thread

Klasa Thread reprezentuje wątek w Javie.

Przykład:

```
Thread t = new Thread(() -> {  
    System.out.println("Kod wykonywany w osobnym wątku");  
});
```

`t.start();`

Metoda `start()` uruchamia nowy wątek. Nie należy mylić jej z `run()`, bo ręczne wywołanie `run()` wykona kod w bieżącym wątku.

Runnable

Runnable to interfejs opisujący zadanie, które może zostać wykonane przez wątek.

Zawiera metodę:

```
public void run()
```

Przykład:

```
Runnable zadanie = () -> {  
    System.out.println("Praca wątku");  
};
```



```
Thread t = new Thread(zadanie);  
t.start();
```

Operacje atomowe

Operacje atomowe wykonują się jako jedna niepodzielna operacja.

Przykład:

```
AtomicInteger licznik = new AtomicInteger(0);
```

```
licznik.getAndIncrement();
```

Metody takie jak:

```
compareAndSet()  
getAndIncrement()  
incrementAndGet()
```

pozwalają bezpiecznie modyfikować dane współdzielone bez klasycznej blokady synchronized.

compareAndSet()

compareAndSet() działa według schematu:

jeśli aktualna wartość jest taka, jak oczekuję,
to zamień ją na nową wartość

Jest to ważna operacja w algorytmach lock-free.

Lock

Lock to bardziej elastyczny mechanizm blokady niż synchronized.

Przykład:

```
Lock lock = new ReentrantLock();
```

```
lock.lock();  
try {  
    // sekcja krytyczna  
} finally {  
    lock.unlock();  
}
```

Blokada chroni sekcję krytyczną przed jednoczesnym dostępem wielu wątków.

Condition

Condition działa podobnie do zmiennych warunkowych.

Pozwala wątkom czekać na określony warunek i być budzonym przez inne wątki.

Przykład mechanizmu:

wątek czeka, aż bufor nie będzie pusty

inny wątek dodaje dane i budzi oczekujący wątek

Kolekcje współbieżne

Java posiada specjalne klasy przystosowane do pracy wielowątkowej, np.:

BlockingQueue

ConcurrentMap

ConcurrentHashMap

Są one zaprojektowane tak, aby wiele wątków mogło bezpiecznie z nich korzystać.

Najważniejsza intuicja

W Javie współbieżność zapewniają zarówno mechanizmy tworzenia wątków (Thread, Runnable), jak i mechanizmy bezpiecznej synchronizacji danych: atomiki, zamki, warunki oraz kolekcje współbieżne.

Pytanie 18: Co można powiedzieć o sposobie przetwarzania i dostępu do zmiennej id?

Kod używa:

```
#pragma omp parallel private(id)
```

Poprawne odpowiedzi

- a. id jest prywatne dla każdego wątku, co oznacza, że każdy wątek ma swoją kopię
- c. Jeśli nie użyto by klauzuli `private(id)`, to id byłoby współdzielone i powodowało błędy
- d. Program może wykonać się poprawnie nawet bez `private(id)`, ale wyniki mogłyby być nieprzewidywalne

Niepoprawne odpowiedzi

- b. id jest wspólne, więc każdy wątek nadpisuje jego wartość — nie, bo w kodzie użyto `private(id)`.
 - e. `private(id)` oznacza, że id jest inicjalizowane domyślnie wartością 0 we wszystkich wątkach — nie, zmienna prywatna nie jest automatycznie zerowana.
-

Pojęcia do notatki

`private(id)` w OpenMP

Klauzula `private(id)` oznacza, że każdy wątek dostaje własną kopię zmiennej id.

Czyli:

Wątek 0 → własne id

Wątek 1 → własne id

Wątek 2 → własne id

...

Dzięki temu wątki nie nadpisują sobie wzajemnie wartości.

Zmienna prywatna

Zmienna prywatna istnieje osobno dla każdego wątku.

W kodzie:

```
id = omp_get_thread_num();
```

każdy wątek zapisuje swój numer do swojej prywatnej kopii id.

Co gdyby nie było `private(id)`?

Gdyby id było zadeklarowane przed regionem równoległym i nie użyto by `private(id)`, wtedy id byłoby zmienną współdzieloną.

Wtedy wiele wątków mogłoby jednocześnie wykonywać:

```
id = omp_get_thread_num();
```

i nadpisywać sobie wartość. Mogłoby dojść do **race condition**, czyli wyniki mogłyby być przypadkowe.

Wartość początkowa zmiennej prywatnej

`private(id)` nie oznacza, że zmienna ma wartość początkową 0.

Wartość początkowa prywatnej kopii jest **niezdefiniowana**, dopóki wątek sam jej nie przypisze.

W tym kodzie nie ma problemu, bo od razu wykonywane jest:

```
id = omp_get_thread_num();
```

Najważniejsza intuicja

`private(id)` sprawia, że każdy wątek ma własne id, więc nie ma wyścigu przy zapisywaniu numeru wątku.

Pytanie 19: Które stwierdzenia dotyczące **OpenMP Task** są poprawne?

Poprawne odpowiedzi:

- a. Dyrektywa `#pragma omp task` umożliwia tworzenie zadań, które mogą być dynamicznie przydzielane do różnych wątków

- d. `#pragma omp taskwait` powoduje, że wątek czeka na zakończenie utworzonych przez siebie zadań
- e. Zagnieżdżanie zadań OpenMP jest możliwe, ale wymaga odpowiedniego zarządzania synchronizacją

Niepoprawne odpowiedzi:

- b. Zadania domyślnie są typu `tied`, co oznacza, że muszą być wykonane przez ten sam wątek, który je utworzył — zdanie jest podchwytliwe. Zadania domyślnie są typu `tied`, ale nie muszą być wykonane przez wątek, który je utworzył. Formalnie zadanie `tied`, jeśli zostanie rozpoczęte przez jakiś wątek i później zawieszone, musi zostać wznowione przez ten sam wątek.
- c. Zadania typu `untied` zawsze są wykonywane natychmiast po ich stworzeniu — nie, `untied` oznacza elastyczne wznowianie zadania przez różne wątki, a nie natychmiastowe wykonanie.

Pojęcia do notatki

`#pragma omp task`

Dyrektywa:

```
#pragma omp task
{
    // kod zadania
}
```

tworzy **zadanie**, które może zostać wykonane przez jeden z dostępnych wątków w zespole OpenMP.

Zadanie nie musi wykonać się natychmiast. Może zostać odłożone i wykonane później przez inny wątek.

Zadania dynamiczne

Zadania OpenMP są przydatne wtedy, gdy praca nie ma prostego kształtu pętli.

Przykłady:

- rekurencja,
- drzewa zadań,
- grafy zależności,
- algorytmy typu dziel i zwyciężaj.

Przykład:

```
#pragma omp task
oblicz_lewa_czesc();
```

```
#pragma omp task
oblicz_prawa_czesc();
```

taskwait

Dyrektywa:

```
#pragma omp taskwait
```

powoduje, że bieżący wątek czeka, aż zakończą się zadania potomne utworzone przez bieżące zadanie.

Przykład:

```
#pragma omp task
zadanie_A();
```

```
#pragma omp task
zadanie_B();
```

```
#pragma omp taskwait
// tutaj mamy pewność, że A i B się zakończyły
```

Zadanie tied

Zadania OpenMP domyślnie są typu **tied**.

Oznacza to, że jeśli zadanie zacznie wykonywać się na danym wątku, to po ewentualnym zawieszeniu musi zostać wznowione przez ten sam wątek.

Ważne: nie chodzi o wątek, który zadanie utworzył, tylko o wątek, który rozpoczął jego wykonywanie.

Zadanie untied

Zadanie typu **untied** może zostać zawieszone i później wznowione przez inny wątek.

Przykład:

```
#pragma omp task untied
{
    wykonaj_prace();
}
```

Nie oznacza to jednak, że zadanie wykona się natychmiast po utworzeniu.

Zagnieżdżanie zadań

Zadania OpenMP mogą tworzyć kolejne zadania.

Przykład:

```
#pragma omp task
{
    #pragma omp task
    {
        zadanie_zagniezdzone();
    }
}
```

Takie zagnieżdżanie jest możliwe, ale trzeba uważać na synchronizację, np. przez `taskwait`, aby nie użyć wyników zanim zadania się zakończą.

Najważniejsza intuicja

OpenMP Task pozwala tworzyć dynamiczne zadania, które mogą być wykonywane przez różne wątki; do czekania na ich zakończenie służy `taskwait`, a różnica między `tied` i `untied` dotyczy tego, który wątek może wznowić zawieszone zadanie.

Pytanie 20: Jakie są różnice między implementacją wielowątkowości za pomocą `Thread` i `Runnable`?

Poprawne odpowiedzi:

- a. Klasa dziedzicząca `Thread` nie może dziedziczyć po innej klasie, co ogranicza jej elastyczność
- b. Implementacja interfejsu `Runnable` pozwala na większą elastyczność kodu, ponieważ klasa może implementować inne interfejsy
- d. `Thread` posiada dodatkowe metody, takie jak `join()`, `interrupt()`, których `Runnable` nie ma

Niepoprawne odpowiedzi:

- c. Metoda `start()` interfejsu `Runnable` automatycznie uruchamia nowy wątek — błędne, bo `Runnable` nie ma metody `start()`. Metodę `start()` ma klasa `Thread`.
 - e. `Runnable` jest bardziej wydajny niż `Thread`, ponieważ działa na wyższym poziomie abstrakcji — błędne. `Runnable` jest bardziej elastyczny projektowo, ale nie oznacza automatycznie większej wydajności.
-

Pojęcia do notatki

Klasa `Thread`

`Thread` to klasa reprezentująca wątek w Javie.

Można utworzyć wątek przez dziedziczenie po `Thread`:

```
class MojWatek extends Thread {
    public void run() {
        System.out.println("Praca wątku");
    }
}
```

```
MojWatek t = new MojWatek();  
t.start();
```

Minusem jest to, że Java nie pozwala dziedziczyć po wielu klasach. Jeśli klasa dziedziczy po Thread, nie może już dziedziczyć po innej klasie.

Interfejs Runnable

Runnable opisuje zadanie, które może zostać wykonane przez wątek.

Przykład:

```
class MojeZadanie implements Runnable {  
    public void run() {  
        System.out.println("Praca zadania");  
    }  
}
```

```
Thread t = new Thread(new MojeZadanie());  
t.start();
```

Ten sposób jest bardziej elastyczny, bo klasa może implementować Runnable, a jednocześnie dziedziczyć po innej klasie.

start() a run()

Metoda start() należy do klasy Thread.

```
t.start();
```

uruchamia nowy wątek i powoduje wykonanie metody run() w tym nowym wątku.

Natomiast ręczne wywołanie:

```
t.run();
```

nie tworzy nowego wątku — wykonuje kod zwyczajnie w bieżącym wątku.

Metody klasy Thread

Klasa Thread ma metody służące do zarządzania wątkiem, np.:

```
join()  
interrupt()  
sleep()  
start()
```

Interfejs Runnable ma zasadniczo tylko metodę:

```
run()
```

Dlatego Runnable opisuje samo zadanie, a Thread reprezentuje mechanizm wykonania tego zadania w osobnym wątku.

Najważniejsza intuicja

Thread to konkretny wątek i ma metody zarządzania nim, a Runnable to samo zadanie do wykonania; Runnable jest zwykle bardziej elastyczny, bo nie blokuje dziedziczenia po innej klasie.